Munich University of Applied Sciences
Faculty of Computer Science and Mathematics



Master Thesis
for obtaining the academic degree of
Master of Science
in course of studies Computer Science

Development and Implementation of a Looking Glass Like Solution for Shibboleth

---

Author:	Tadashi Toyokawa
Matriculation number:	27740221
Submission Date:	4. November 2026
Supervisor:	Prof. Dr.-Ing. Thomas Schreck
Second Supervisor:	Prof. Dr.-Ing. Erik Krempel
Advisor:	Erwin Kupris

I confirm that this masterarbeit is my own work and I have documented all sources and material used.

Munich, 4. November 2026 Tadashi Toyokawa

Acknowledgments

I would like to express my deepest gratitude to my advisor, Erwin Kupris, my supervisor Prof. Dr.-Ing. Thomas Schreckand my second supervisor Prof. Dr.-Ing. Erik Krempel, for their invaluable guidance, insightful feedback, and continuous support throughout this research. Furthermore, I extend my appreciation to my family and friends for their encouragement and support during this journey. Their motivation and belief in my abilities have been a constant source of strength.

Abstract

In a federated identity infrastructure, a single identity provider authenticates users on behalf of many service providers and thereby becomes the trust anchor of the entire federation. A compromised or misconfigured provider does not affect one application but can undermine the security of every service relying on it. Despite this exposure, systematic security monitoring at the application layer of such providers remains largely absent, and existing approaches in this domain focus on operational availability rather than on security.

This thesis presents a monitoring system that observes a Shibboleth identity provider from within its own application layer. The system draws on three sources that differ in what they can observe, namely the audit log written at the end of a transaction, the configuration and metadata declared for each service provider, and the runtime state accessible through interceptors executing inside the authentication flow. Records from these sources are pseudonymised at their origin, transmitted to a backend located outside the trust boundary of the provider, correlated with one another and evaluated against a catalogue of fifteen detection rules. The rules are derived from vulnerabilities and attack classes documented in a systematic survey of federated identity security and from the security recommendations of the OASIS SAML Security and Privacy Considerations.

The evaluation addresses three questions, namely which data can be collected, how reliably it can be correlated, and what the resulting system costs. The correlation reaches a coverage of 100 % over 7,236 reports. Where the audit log records the identifier of the request, it assigns every report unambiguously, independent of the login density. The heuristic strategies that take over where the identifier is missing remain free of incorrect assignments up to a quantified login density, beyond which they can no longer resolve the order of events. All fifteen rules were verified against a real search index and six additionally through the complete pipeline from a browser login, four of them up to the resulting alert. In the asynchronous transmission mode, the end-to-end login latency did not differ measurably from a deployment without interceptors, with an upper bound of 17 ms for the 95% confidence interval, below five percent of the median login duration. The processor load of the IdP increased by about seven percentage points of one core, whereas blocking transmission added about 60 ms to each login.

The work also identifies the boundaries of the chosen observation point. Attack classes located at a different system layer, involving a different party, or observable only through their preconditions remain outside its reach, and a provider abusing its own position cannot be detected by a system executing inside it. Beyond these results, each of the three evaluations uncovered a defect in the infrastructure or the tooling of this work that operation itself had not revealed, which illustrates the premise underlying the work, namely that a system appearing to function is not thereby known to function.

Contents

1 Introduction **1**

2 Background **3**
2.1 Technical Foundation of Federated Identity Management 3
2.2 SAML 4
2.2.1 SAML Assertions 4
2.2.2 XML-Signature and Encryption 5
2.2.3 SAML Bindings 6
2.2.4 Web Browser SSO Authentication Process 6
2.2.5 SAML Metadata 8
2.3 Shibboleth 8
2.3.1 Profile Flows and Interception Points 9
2.3.2 The Context Tree 9
2.3.3 Audit Logging 10
2.3.4 Configuration 10

3 Related Work **11**
3.1 Security Analyses of SAML and Shibboleth 11
3.2 Why Monitor from the Inside? 11
3.3 Why SIEM Functionality Is Not Enough 13
3.4 Derivation of the Rules 15

4 System Design **17**
4.1 Requirements for the System 17
4.2 Overview 18
4.3 Data Aggregation Layer 19
4.3.1 The Audit Logs 19
4.3.2 Metadata and Relying Party File 20
4.3.3 Interceptor 21
4.4 Transport Layer and Trust Boundary 23
4.5 Correlation and Detection Layer 25
4.5.1 Correlation 25
4.5.2 Detection 26
4.5.3 Severity Classification 28
4.6 Visualisation Layer 29
4.7 Design Decisions and Rejected Alternatives 30

5 Implementation **33**
5.1 Data Aggregation Layer 33
5.2 Transport Layer 35

5.3 Correlation and Detection Layer 36
5.3.1 Storage 36
5.3.2 Correlation 37
5.3.3 Detection 38
5.4 Visualisation Layer 40

6 Evaluation **43**
6.1 Test Setup 43
6.2 Correlation 43
6.2.1 Method and Datasets 44
6.2.2 Defects Identified during the Evaluation 46
6.2.3 Results of the Evaluation 46
6.2.4 Coverage and Operating Limits of the Correlation 51
6.3 Detection 54
6.3.1 What the Figures Represent 54
6.3.2 Test Layers 55
6.3.3 Coverage per Rule 56
6.3.4 A Defect Visible Only through the Full Pipeline 57
6.3.5 A Defect Visible Only against Real Persistence 57
6.3.6 A Systemic Class of False Negatives 58
6.3.7 Consistency between Outbound Report and Audit Entry 58
6.3.8 Coverage of the Documented Attack Space 59
6.3.9 Thresholds 60
6.3.10 Fidelity of the Rules and Defects Uncovered by Testing 60
6.4 Performance Overhead 61
6.4.1 Measurement Sources 61
6.4.2 Experimental Design 61
6.4.3 Descriptive Results 62
6.4.4 Paired Analysis of the Latency 63
6.4.5 CPU Utilisation 64
6.4.6 Position Drift as a Methodological Finding 65
6.4.7 Resolution of the Measurement Environment 65
6.4.8 Correction of the Analysis Script 66
6.4.9 Latency and Processor Cost of the Interceptors 66
6.5 Threats to Validity 67
6.5.1 Measurement Environment 67
6.5.2 Traffic and Scope 67
6.5.3 Statistical Basis 68
6.5.4 What the Detection Figures Establish 68
6.5.5 Derived but Not Validated 69
6.5.6 Known Gaps 69

1 Introduction

In today's increasingly digital world, where most online services require users to create an account to access them, users are forced to create and memorise a large number of different credentials, or are even forcibly encouraged to reuse old passwords on multiple services. These online profiles are highly sensitive, as they are typically associated with the user's personal identity, including details such as their email address, phone number or even official government documentation like a personal ID. More often than not, this profile is protected by a single password created by the user. This dominant method of password-based authentication, which is responsible for verifying and authenticating users online, poses a significant risk to companies and other large organisations. One way in which malicious attackers take advantage of this vulnerable authentication method is by using large databases of leaked user credentials (often including email addresses and corresponding passwords) to authenticate at Service Providers (SPs) as the affected user. This attack method is called credential stuffing. Verizon reports that credential abuse was the leading initial attack vector in 22% of the 12,195 confirmed data breaches in the 2025 report [58].

To reduce the number of places where credentials are created and stored, Federated Identity Management (FIM) was developed. FIM lets users access several SPs with a single set of credentials that is managed by an Identity Provider (IdP). FIM consolidates authentication behind a single trusted authority called the IdP. When a user attempts to gain access on the SP in FIM, the user's browser is redirected to the IdP. The user authenticates with the IdP, which then confirms the user's identity to the SP [52].

Shibboleth, developed by Internet2, is one of the most widely deployed open-source implementations of this model in the research and education sector [49]. Under this model, a user authenticates with their organisation's IdP once. The IdP then issues cryptographically signed SAML assertions that SPs accept as proof of identity. This architecture is widely deployed in higher education and research federation such as DFN-AAI [17], where a single IdP serves dozens of SPs simultaneously.

This architectural property introduces a critical consequence: the IdP becomes a single point of trust for the entire federation [53]. A compromised, misconfigured or vulnerable IdP does not affect one application. It can undermine the security of every relying SP. A notable example is the Sunburst supply chain attack [16]. As Hartl and Derek note, after the initial compromise the attackers obtained the trusted certificate used for signing SAML assertions and used it to access all services within the targeted enterprise [20]. Since tokens forged with this certificate carry the organisation's own trusted signature, the resulting anomalies may go unnoticed [26]. Beyond targeted attacks, SAML deployments are exposed to weaknesses in the protocol and in its implementations. Somorovsky et al. found XML Signature Wrapping (XSW) vulnerabilities in eleven of fourteen SAML frameworks, including Shibboleth, which allow an attacker to inject forged claims that the SP accepts once the signature has been validated [56], and the attack has since been reproduced in other OpenSAML-based setups [23]. In a formal analysis of Shibboleth, Sommer identified

three further attacks, namely Response Injection, Leaked Assertion Artifact and IdP Spoofing, which let an attacker log a victim into an account controlled by the attacker or reuse a leaked artifact to log in as the victim. The Shibboleth developers attribute the first of these to the SAML standard itself rather than to the implementation [55].

Despite this elevated risk profile, systematic security monitoring at the application layer of Shibboleth IdP deployments remains largely absent. Existing approaches such as AAIEye [57] focus primarily on operational availability, leaving security-relevant aspects unaddressed. The research literature reflects a comparable gap. A survey of security analyses in federated identity management finds Shibboleth to be among the least examined protocols and raises the question of whether this reflects a stronger security posture or merely a lower level of attention [52]. Formal analyses of SAML likewise remain scarce despite its deployment across government, enterprise and education [20].

This thesis addresses this gap by proposing a security-oriented monitoring system that operates directly within the Shibboleth application layer. It makes application-layer security parameters and authentication workflows visible that are otherwise inaccessible from outside the application. Rather than testing the deployment actively, the system observes it passively. This principle has been applied to the eduroam infrastructure by analysing its network traffic [39], whereas this work applies it inside the application, at the application layer of the IdP. The system thereby provides federation operators with a continuous view of the security posture of their authentication infrastructure. The title of this thesis refers to this idea of a view into an otherwise opaque infrastructure. The system is not a looking glass in the sense of the network tools of the same name, which offer queries on a device, but a monitoring system for the operator of the IdP.

Three research questions guide this work:

- RQ1 Which security-relevant data can be collected at the application layer of a Shibboleth IdP, and through which mechanisms are these data accessible?
- RQ2 How can these data be correlated in order to detect configuration weaknesses and anomalies in the authentication process?
- RQ3 What measurable overhead does the resulting system impose on the monitored IdP?

The remainder of this thesis is structured as follows.

- Chapter 2 introduces the technical foundations of FIM, SAML and the Shibboleth IdP.
- Chapter 3 positions this work within existing research on application layer intrusion detection and delimits it from network and host based monitoring systems.
- Chapter 4 presents the architecture of the proposed system.
- Chapter 5 describes its implementation, divided by layer.
- Chapter 6 evaluates the system with respect to the three research questions.
- Chapter **??** interprets the results and discusses the limitations of the approach.
- Chapter **??** summarises the findings.

2 Background

This chapter explains the key concepts and terminology relevant to this thesis and establishes the technical foundation required for the design of the proposed monitoring tool. It is limited to the aspects on which the later chapters build, and consequently covers neither the full range of the SAML specification nor the complete configuration surface of the Shibboleth IdP.

2.1 Technical Foundation of Federated Identity Management

As introduced in Chapter 1, FIM allows users to access the resources they need with a single digital identity and provides Single Sign On (SSO) functionality across the applications of an organisation. The immediate benefit is convenience, since users no longer need to create and remember a separate set of credentials for every service. The more consequential benefit lies elsewhere. Because credentials exist in one place rather than in every participating service, the number of systems that store and verify them is reduced, since the RP neither verifies the user's authenticators itself nor needs to issue its own [33]. The strength of authentication is consequently also determined in one place. An RP can require a minimum authentication assurance level, which the IdP fulfils and reports in the assertion [33], and from the second level upwards this requires two distinct authentication factors [32]. Concentrating authentication at one well-managed site also focuses administrative effort and makes multi-factor authentication more acceptable to users, who encounter it less often [18]. Administration follows the same pattern, since accounts are created, modified and revoked centrally rather than in each service individually.

This centralisation introduces a new risk in return. FIM rests on a relationship of trust between organisations, and that trust depends entirely on the security of the IdP. Concentrating credentials in one place also concentrates the consequences of a compromise, since an attacker who gains control of the IdP can potentially authenticate as any of its users at the services that rely on it [53].

A compromise of a primary IdP could potentially grant broad unauthorised access across the entire federation. This characteristic requires both the IdP and SP to maintain a high level of security [33]. The policies defined for each SP in the IdP must accurately translate to the rules at the corresponding SP. The configuration of the IdP and that of each SP must remain consistent with one another. Certificates, keys and protocol settings are maintained separately on both sides, and a mismatch between them can weaken the security of a transaction without either party noticing. To better understand the interaction within a federated identity system, the roles and responsibilities of the IdP and SP are described in the following.

A federation consists of two kinds of participants. The IdP authenticates users and issues statements about them [38]. The SP offers the resource a user wants to reach

and relies on those statements instead of authenticating the user itself. Neither role is meaningful without the other, and their interaction follows a fixed pattern.

When a user requests a protected resource, the SP does not check credentials. It determines which IdP is responsible for the user and redirects the browser there together with an authentication request [38]. The IdP authenticates the user by whatever means the deployment provides, and returns a signed statement asserting that the authentication took place and describing the user by a set of attributes. The SP verifies the signature, evaluates the statement and grants or denies access on that basis. The credentials themselves never leave the IdP.

This division has a consequence for how a federation is configured. Because the IdP serves many SPs, it does not treat them uniformly. It maintains a separate configuration for each partner, specifying for instance whether the statement is to be encrypted, which authentication methods are permitted, and which attributes are released. The SP maintains its own configuration in turn, stating what it expects to receive and what it will accept.

Both sides therefore hold configuration that describes the same relationship, and both are maintained independently. What binds them together is the metadata each party publishes about itself. This includes the endpoints at which it can be reached, and the certificates by which its statements can be verified. Trust within a federation rests on these metadata being current and correctly exchanged [14]. Whether the two configurations remain consistent with one another, however, is not something the protocol itself verifies.

2.2 SAML

The Security Assertion Markup Language (SAML) is an XML-based standard developed by the Organization for the Advancement of Structured Information Standards (OASIS) for the exchange of authentication, authorisation, and other security-related information between different parties. SAML is a fundamental component of FIM and is widely used to enable SSO across organizational and security boundaries [38]. The second major version, SAML 2.0, was published by OASIS in March 2005 [34]. SAML 1.1 had already gained widespread adoption in federated identity environments [35]. However, several competing approaches existed at the time, including the Identity Federation Framework (ID-FF) developed by the Liberty Alliance and the Shibboleth architecture [34]. SAML 2.0 consolidated several of these approaches into a common standard and introduced additional capabilities, particularly with regard to federation, interoperability, and privacy.

2.2.1 SAML Assertions

SAML assertions serve as security tokens that convey security-related information about a principal, such as a user, system, or service. An assertion contains statements made by an issuer about the principal, which may include authentication, authorisation, and attribute information. Since these statements can have security-critical implications, their authenticity and integrity must be ensured [13]. Therefore, the SAML element can be protected by a digital signature following the XML Signature standard, allowing a

relying party to verify that the assertion was issued by the stated issuer and has not been modified [56].

Each assertion contains metadata that defines its origin and validity. The `Issuer` element identifies the entity that issued the assertion, while the `IssueInstant` attribute specifies the time at which it was issued. Furthermore, every assertion is assigned a unique identifier that must not be reused. Once an assertion has been issued, the issuer has no direct control over its subsequent use. Consequently, a consumer of an assertion is expected to enforce its specified validity conditions, including any validity interval and applicable `OneTimeUse` condition.

2.2.2 XML-Signature and Encryption

To protect the integrity and authenticity of SAML assertions, XML signatures can be used to cryptographically bind the assertion to its issuer. The W3C XML Signature standard defines how signatures over XML data are created, encoded and verified [1]. A signature is defined by the `Signature` element and can be applied to an entire SAML assertion or to selected parts of an XML document [13, 56].

Depending on the location of the signature relative to the signed data, XML signatures can be categorised into three types [1]. An enveloped signature is contained within the XML element that it signs. In contrast, an enveloping signature contains the signed data within the signature itself. A detached signature is kept apart from the signed data, which it neither contains nor is contained in, and refers to that data by reference [1]. In the context of SAML, enveloped signatures are commonly used to sign SAML assertions and protocol messages.

XML Encryption provides confidentiality for sensitive information contained in SAML messages. Instead of transmitting the original XML content in plaintext, the protected content is replaced by an `EncryptedData` element containing the resulting ciphertext [22]. The encryption therefore enables selective and persistent confidentiality of individual elements within an XML document. This allows, for example, sensitive attributes contained in a SAML assertion to be encrypted without requiring the entire message to be encrypted.

Although XML Signature and XML Encryption provide important security mechanisms, their use and configuration depend on the respective SAML profile, deployment, and implementation. Therefore, SAML is not a security protocol in itself, but rather a framework whose optional features can be combined in various ways. The resulting security properties depend on the chosen combination. Hartl and Derek make this explicit by deriving eight protocol variants from the optional features of a single use case and analysing each separately [20]. Consequently, the security properties of a SAML-based system cannot be derived solely from the existence of these mechanisms in the standard. Incorrect configuration or implementation may result in insufficient protection, such as assertions that are not appropriately signed or sensitive information that is transmitted without encryption.

2.2.3 SAML Bindings

SAML bindings define how SAML protocol messages are transported using underlying communication protocols. They are particularly relevant for the Web Browser SSO profile, where the user's web browser acts as an intermediary between the IdP and SP. The SAML specification defines several bindings for different communication scenarios. In a typical SP-initiated Web Browser SSO process, the HTTP-Redirect and HTTP-POST bindings are particularly relevant [12].

2.2.3.1 HTTP-Redirect Binding

The HTTP-Redirect binding is designed for scenarios in which SAML requester and responder communicate through an HTTP user agent acting as an intermediary. In the SP-initiated SSO flow, the SP acts as the SAML requester and the IdP as the SAML responder. The authentication request is encoded and transmitted as part of the URL query string of an HTTP redirect. The user's browser follows the redirect and thereby forwards the request to the IdP.

Since the message is transported through the URL using the HTTP GET method, the binding is particularly suitable for relatively small SAML messages, such as an authentication request. The HTTP-Redirect binding therefore provides the mechanism by which the SP can initiate the authentication process without requiring a direct communication channel between the SP and IdP.

2.2.3.2 HTTP-POST Binding

The HTTP-POST binding is likewise designed for communication between a SAML requester and responder through an HTTP user agent. In the return direction of the typical SP-initiated SSO flow, the IdP transmits the SAML response to the SP. The SAML response is embedded in an HTML form and sent to the SP using the HTTP POST method. The user's browser acts as an intermediary by submitting the form to the SP.

Compared with the HTTP-Redirect binding, HTTP-POST is better suited for larger SAML messages because the message is not encoded into the URL. This makes it particularly suitable for SAML responses containing assertions and associated security-related information.

2.2.4 Web Browser SSO Authentication Process

The SAML Web Browser SSO profile enables a user to authenticate to an IdP and subsequently access a protected service provided by an SP. The process is mediated by the user's web browser, which forwards SAML messages between the SP and IdP.

The process begins when a user requests access to a protected resource at the SP. If the user does not have an established session, the SP generates a SAML authentication request and redirects the user's browser to the IdP. The IdP processes the request and authenticates the user using the configured authentication mechanism. Upon successful

authentication, the IdP generates a SAML response containing a SAML assertion with security-related statements about the authenticated principal.

The assertion can be digitally signed by the IdP to provide authenticity and integrity. The SAML response is then returned to the user's browser, which forwards it to the SP. The SP validates the received SAML response and assertion, including the digital signature and the applicable validity conditions. If the assertion is considered valid, the SP evaluates the contained statements and attributes to determine whether the user is authorised to access the requested resource. If the authorisation requirements are satisfied, the SP grants the user access to the protected service [38]. Figure 2.1 illustrates the described authentication process.

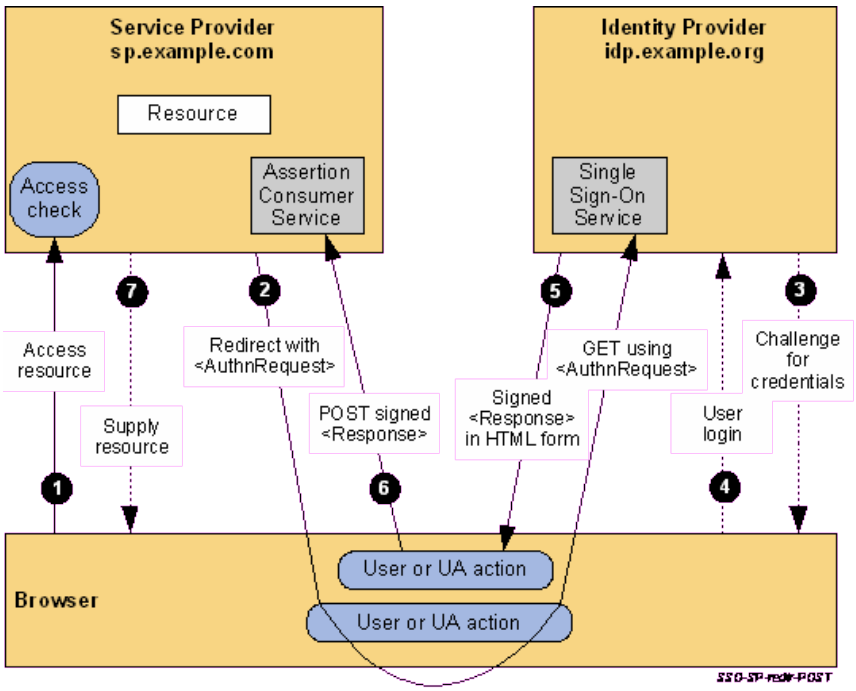


Figure 2.1: *SP-Initiated SSO with Redirect and POST Bindings [38]*

The use of the user's web browser as an intermediary introduces an important security consideration. The SAML messages exchanged between the IdP and SP are transmitted through the user's browser and therefore pass through a communication path that is not directly controlled by either party. Neither the IdP nor the SP can assume that the messages remain unchanged while being forwarded. Consequently, the security of the protocol cannot rely solely on the communication channel. Instead, mechanisms such as digital signatures, message identifiers, validity conditions, and audience restrictions are used to ensure the authenticity and integrity of SAML messages and to prevent their misuse.

An important security property of this process is the prevention of message replay. SAML requests and assertions contain unique identifiers that allow the SP and IdP to assign messages and detect previously processed messages. In particular, an assertion's

unique identifier must not be reused, and the recipient is expected to enforce the applicable validity conditions and replay-related constraints.

2.2.5 SAML Metadata

SAML metadata provides a standardised way for SAML entities to describe their configuration and capabilities to other participating entities. It enables IdPs and SPs to obtain information required for establishing communication and processing SAML messages without requiring every configuration parameter to be exchanged manually. The SAML metadata specification organizes this information around roles supported by an entity, which are represented using role descriptors within an `EntityDescriptor` element [14].

For an IdP, the `IDPSSODescriptor` describes information relevant to its role as a SAML IdP. Similarly, an SP can be described using an `SPSSODescriptor`. These descriptors contain information such as supported SAML protocols, communication endpoints, supported bindings, and cryptographic keys. For example, an SP metadata document can specify its Assertion Consumer Service (ACS) endpoint and the bindings supported for receiving SAML responses, while an IdP can specify its SSO Service endpoints and the corresponding supported bindings.

Cryptographic information contained in metadata is particularly important for the security of SAML communication. Metadata can contain certificates and public keys used by participating entities for verifying digital signatures or encrypting SAML messages. Consequently, the metadata configuration directly influences how SAML messages are validated and which endpoints and cryptographic keys are considered valid by an implementation.

The security of the metadata itself is therefore an important aspect of a SAML deployment. If an entity accepts manipulated metadata, an attacker could potentially influence endpoints, supported bindings, or cryptographic keys used during SAML message processing. SAML metadata can itself be protected using digital signatures, allowing an entity to verify the integrity and origin of the metadata before using it for configuration or trust decisions.

2.3 Shibboleth

Shibboleth is a free and open-source web single sign-on software suite that implements SAML and other standards for FIM. It is widely used in federated authentication environments and places particular emphasis on privacy and decentralised identity management. The Shibboleth Consortium provides separate software components for the IdP and SP, enabling organizations to participate in federated identity infrastructures [49].

For the scope of this thesis, the Shibboleth IdP is of particular relevance, as it is responsible for authenticating users and issuing SAML assertions to participating SPs. The IdP is implemented as a Java-based web application and is typically deployed using a Java servlet container such as Apache Tomcat. User identities and associated attributes are commonly managed through an external directory service, such as a Lightweight Directory Access Protocol (LDAP) server, which provides the IdP with the information required

for authentication and attribute release. It builds on the OpenSAML library for the construction and validation of SAML messages, which is relevant because vulnerabilities reported for OpenSAML can affect Shibboleth deployments as well [18].

2.3.1 Profile Flows and Interception Points

The IdP processes each request as a workflow built on Spring Web Flow, a framework in which a sequence of processing steps is described declaratively rather than in program code. A flow consists of states that are traversed in order, and a state may delegate to a subflow, that is a self-contained flow which performs a defined task and returns control to its caller. Every top-level flow of the IdP implements one of the supported protocol interfaces, referred to as profiles, such as SAML 2 SP-initiated SSO. Recurring tasks within such a profile, among them the authentication of the user, are carried out by subflows [9].

To allow developers to extend this behaviour without copying or editing the internal flow definitions, the IdP provides a set of defined hooks. A subflow attached to such a hook is called an interceptor, and three hooks exist:

1. inbound message processing
2. post-authentication processing and
3. outbound message processing

The first and third are available in any profile flow, whereas the second applies only to profiles that actually authenticate a user [9]. An interceptor is registered as a bean and enabled by adding its flow identifier to the corresponding list property of a profile configuration in `conf/relying-party.xml`. No modification of the IdP source code is required.

The choice of hook determines what an interceptor can observe. An inbound interceptor sees the incoming message context but not the outcome of authentication, since that has not yet taken place. An outbound interceptor sees the state of the tree produced by the profile execution, including the outgoing message. The post-authentication hook runs in the middle of processing, after the user has been authenticated but before the outgoing message is constructed. It can therefore observe both the incoming request and the authentication result, but not the response that is issued afterwards.

2.3.2 The Context Tree

While a flow describes the sequence of processing steps, the state of the request being processed is held separately in a structure referred to as the context tree. Its root is the `ProfileRequestContext`, which is created when a request arrives and accompanies it through the entire profile execution. Attached to this root are child contexts, each holding one aspect of the request, so that the state grows as processing advances rather than being assembled in a single object at the outset.

Three of these child contexts are relevant in the following. The `InboundMessageContext` holds the message as it was received, the `RelyingPartyContext` identifies the SP the request originates from and records whether that identity could be verified, and the

`SubjectContext` holds the authenticated subject once authentication has completed [9]. The last of these illustrates the incremental nature of the structure, since it does not exist before the user has been authenticated.

2.3.3 Audit Logging

The IdP writes one line per completed transaction to `idp-audit.log`. The set of fields is not fixed but defined by the operator in `conf/audit.xml`, so the information available in the log depends on the configuration of the individual deployment. In the default configuration, authentication attempts that do not result in a completed transaction, in particular failed password validations, produce no entry in this log. The IdP can additionally record every authentication attempt, but this feature is disabled by default, and when enabled its records enter the same audit stream unless they are routed to a separate destination [4].

The format of a log line is defined per profile in the *shibboleth.AuditFormattingMap* of `conf/audit.xml`. A format consists of formatting tokens, each of which is replaced by a value from the context tree of the transaction when the line is written, for instance the time of the event, the identifier of the inbound message or the address of the client. Some values are written in an abbreviated form, which the IdP derives from a replacement map (*shibboleth.AuditFieldReplacementMap*), and further tokens can be added by the operator [4].

2.3.4 Configuration

Three configuration files are relevant in the following. `relying-party.xml` holds the per-partner configuration described in Section 2.1, including which profiles are enabled and which security settings apply [11]. `metadata-providers.xml` declares where the IdP obtains metadata for its partners [7]. `attribute-filter.xml` governs which attributes are released to which relying party [3].

The settings of `relying-party.xml` apply to all partners by default and can be overridden for a single partner, so that one partner is configured differently from the others. They include the authentication flows permitted for a partner and whether responses and assertions are signed or encrypted [11].

3 Related Work

Existing research and implementations were examined to identify approaches relevant to the development of a monitoring system for an IdP operating at the application layer. The primary objective was to identify solutions that address the overarching goal of this thesis and to analyse their respective limitations and areas of applicability.

These related works provide valuable insights and a foundation for this thesis by identifying existing approaches, their areas of coverage, and gaps that remain relevant to the considered scenario. The most relevant studies and projects are presented and analysed in the following sections.

3.1 Security Analyses of SAML and Shibboleth

The security of SAML-based federations has been examined from two directions. Empirical studies analyse existing implementations, while formal approaches reason about the specification itself.

Somorovsky et al. examined fourteen SAML frameworks and found critical XSW vulnerabilities in eleven of them [56]. Their analysis is confined to the processing of assertions at the SP, and it does not consider what information is available at the IdP during or after such an attack. Kaur and Bansal transferred this line of work to a cloud setting by reproducing an XSW attack in their own OpenSAML-based SSO environment and rating its severity with CVSS, which fell from 6.2 to 2.7 once a mitigation was applied [23].

A second line of work approaches the problem formally. Sommer analyses the Shibboleth implementation with respect to authentication, session integrity and logout consistency [55], while Hartl and Derek analyse the SAML specification itself [20]. The latter derive eight protocol variants from the optional features of a single use case and prove the security properties of each separately. Their premise is that SAML is not a security protocol in itself, but a framework whose optional features can be composed in different ways.

Both approaches establish which configurations are secure under a given set of assumptions. Neither verifies whether a deployed system actually satisfies those assumptions. Formal analysis therefore answers which configuration is secure, whereas this work observes which configuration is in effect.

3.2 Why Monitor from the Inside?

As discussed in the Chapter 1, the proposed monitoring system operates directly within the application layer of the Shibboleth IdP. Its objective is to detect attacks based on events and behaviour observable from within the application.

The underlying concept is not new. The OWASP AppSensor project describes an approach in which intrusion detection and response capabilities are integrated directly

into an application [37]. In contrast to network-based security mechanisms, application-layer monitoring can make use of information about the application's internal state and semantics.

A network firewall primarily controls network communication, while a network-based Intrusion Detection System (IDS) has limited visibility into application-specific behaviour. A Web Application Firewall (WAF) can provide additional protection against generic application-layer attacks, such as SQL injection, but typically has limited knowledge of the internal logic and state of a specific application. Consequently, application-integrated detection mechanisms can complement network- and perimeter-based security controls by analysing application-specific events and behaviour [37].

This application-level perspective also provides a basis for classifying the proposed system within an established security taxonomy. In the following, the proposed system is classified using the taxonomy for self-protecting software systems introduced by Yuan and Malek [61]. Two dimensions of this taxonomy are particularly relevant to the scope of this work: the `Architecture Level` and the `Self-Protection Level`.

The `Architecture Level` describes the layer at which a security mechanism operates and distinguishes between Network, Host, Software/Application, and Services. This dimension allows the observation position of different security mechanisms to be compared. Network-based IDS, such as Suricata and Snort, operate primarily at the network level, whereas Endpoint Detection and Response (EDR) solutions operate at the host level. The proposed system operates at the Software / Application level, as it observes events directly within the Shibboleth IdP. The distinction between these systems therefore lies primarily in the layer at which security-relevant information becomes observable.

While the Architecture Level describes where the system operates, the `Self-Protection Level` describes the degree of self-protection according to the MAPE process of Monitor, Analyze, Plan, and Execute. The proposed system fully covers the *Monitor & Detect* level by collecting and detecting security-relevant events within the IdP. The *Analyze & Characterise* level is partially addressed through event correlation and severity classification. In contrast, *Plan & Prevent* is outside the scope of this work. The system generates alerts based on detected events but does not autonomously modify the system state or take preventive countermeasures.

This limited level of self-protection is intentional and follows from the passive design principle of this work. The proposed system follows the principle of passive observation, in which security-relevant events are made observable without interfering with the operation of the monitored system. Rieckers applies the principle of passive observation to network traffic, capturing the RADIUS and EAP-TLS traffic of the eduroam infrastructure to analyse the TLS parameters that clients offer [39]. In captured traffic, information that exists only inside the application, such as the structure of a SAML assertion or the configuration of a SP, is not visible. This work adopts the principle and applies it inside the Shibboleth IdP itself. Extending the system with automatic response mechanisms would contradict this passive observation principle. Furthermore, erroneous automated responses could negatively affect the availability or operation of the IdP and thereby potentially impact multiple SPs that rely on it.

The principle of passive observation is also reflected in existing application-level monitoring approaches. One relevant architectural precedent is BlackWatch by Hall et al. [19]. BlackWatch applies a similar architecture to generic web applications. Detection points are placed directly within the application to identify security-relevant events. These events are collected by the separate BlackWatch application, where they are analysed using rule-based and reputation-based mechanisms. The results are then made available through a web-based reporting interface.

The separation between the monitored application and the analysis component allows the detection and analysis functionality to be decoupled from the application itself. A key challenge identified by Hall et al. is the integration and configuration of detection points within the application. Developers must identify suitable locations where suspicious behaviour can be detected and implement the corresponding detection logic [19].

A domain-specific precedent is provided by Suvanto's AAIEye, a monitoring system for Shibboleth- and SAML 2.0-based federations [57]. Motivated by the lack of effective monitoring solutions for authentication and authorisation infrastructures, AAIEye deploys probes alongside the monitored IdP and SP components. These probes collect two categories of information that closely resemble the sources used in this work: runtime authentication events and federation metadata.

However, the evaluation applied to this data differs fundamentally. The usage statistics probe records an authentication event only as a data point for a login count, without assessing whether the event itself is suspicious. The metadata probe verifies only the currency of a metadata file, through a checksum comparison against a reference copy, not its security-relevant content such as certificate validity or endpoint configuration. Suvanto notes that problems in federation operation are often caused by configuration mistakes or changes that cannot currently be easily detected [57]. This is precisely the gap addressed in this work. The same categories of data that AAIEye already collects for operational purposes are evaluated here for their security semantics instead.

A complementary perspective is provided by Martín et al., who propose behavioural anomaly detection at the SP rather than the IdP [28]. Their argument is that delegating authentication to an IdP does not transfer the entire security responsibility, and that the relying party can contribute to detection even after authentication has succeeded. The IdP side is explicitly outside their scope, and protocol-level information such as assertion structure, attribute release policies or SP metadata is consequently not considered. Taken together with the present work, the two perspectives address opposite ends of the same federation.

3.3 Why SIEM Functionality Is Not Enough

A Security Information and Event Management (SIEM) primarily operates on logs and security-relevant events. This is not a criticism, but a description of its basic operating principle. Two limitations follow from this principle. First, information that is not represented in an event or log is not available to the SIEM for analysis. Second, the SIEM

generally relies on the information provided by its data sources and therefore cannot independently verify information that is incorrect or has been manipulated at the source.

These limitations do not diminish the value of SIEM systems, but they define the boundary within which they operate. The collection, storage, and correlation of log data used in this work therefore correspond to established SIEM functionality and do not constitute an original contribution. OWASP A09 explicitly recommends centralised logging and monitoring and identifies solutions such as the ELK stack as possible tools for addressing logging and monitoring vulnerabilities [36].

The question therefore shifts from how events are analysed to how their underlying sources can be verified. Wazuh is particularly relevant in this context, as it combines log analysis with file integrity monitoring and security configuration assessment, thereby addressing both event-based analysis and the verification of system state [59]. However, these mechanisms primarily address configuration and file state rather than the relationship between configuration and individual runtime events. Security configuration assessment evaluates system configurations against defined policies, while file integrity monitoring detects changes to monitored files. Neither mechanism directly establishes whether a particular runtime event is consistent with the configuration state that was active when the event occurred.

A further concern arises when considering the integrity of the log sources themselves. Graves discusses this concern for Shibboleth under the STRIDE category of repudiation. Prompted by unusual entries reported on the Shibboleth mailing list, he concludes that log entries can be injected that lead the administrator astray, and suggests keeping the Apache logs, which contain the actual browser requests, separate from the Tomcat and IdP logs, which may themselves be subverted [18]. CWE-117 describes the underlying mechanism: input that is written to a log without neutralisation allows an attacker to insert forged entries and thereby mislead subsequent analysis [30]. An attacker who has gained access to the system can go further and alter entries that have already been written. OWASP A09 lists insufficient protection of logs against tampering as a weakness and recommends audit trails with integrity controls to prevent tampering or deletion [36]. AppSensor dedicates a detection point to this issue (IE6), which involves modifying an existing log entry [37]. Snodgrass et al. assume such an attacker in their threat model, who after a successful penetration has access to the database, the operating system and the hardware [54].

This makes the correlation of independent event sources more than a technical convenience. It provides an additional means of identifying inconsistencies between records of the same event. A system that relies solely on log analysis can compare multiple log sources, but if all sources are stored as files on the same system, they remain exposed to an attacker with sufficient file-system access.

Cryptographic approaches provide a stronger solution to this problem. Schneier and Kelsey, as well as Snodgrass et al., describe mechanisms that make tampering with stored logs cryptographically detectable [42, 54]. However, these approaches require changes to the logging or storage architecture and may depend on external trusted components. Snodgrass et al. report an overhead of up to 16 percent for their approach [54].

The approach taken in this work is different. Instead of modifying the existing logging infrastructure, two independently generated records of the same event are compared. The difference is therefore not based on a new trust boundary, but on the temporal and architectural separation between the records. A log file may remain accessible to an attacker with file-system access for an extended period, whereas a record generated within the application process itself exists only for the duration of the corresponding processing before it is transmitted away. This does not make a record produced inside the application process immutable: an attacker controlling the JVM process could also manipulate it. Its purpose is instead to provide a second, independently generated observation of the same event.

The remaining question is whether such application-level detection is already sufficiently covered by existing frameworks. Of the works considered, AppSensor is conceptually closest to this work. However, its detection model is based on different assumptions about the monitored application [37].

AppSensor detection points primarily monitor the behaviour of a user within an application session. They can detect events such as tampered cookies, modified parameters, or injection attempts. The framework therefore asks whether a user's behaviour is suspicious. It does not provide a mechanism for comparing an observed runtime event with a configuration or security state declared by a third party.

This limitation follows from the architectural assumptions of the framework. AppSensor is designed for applications in which the application itself is the primary authority over the user's interaction. An IdP, in contrast, performs authentication and issues identity information on behalf of other entities. Consequently, security-relevant conditions may depend not only on the IdP's own runtime behaviour, but also on configuration and trust relationships involving its federation partners.

This creates a detection category that is not directly represented in the AppSensor model: comparing an observed runtime event against the configuration that should govern that event. One AppSensor detection point, however, is directly applicable. IE6, *Violation of Security Log Integrity*, addresses the integrity of security logs and cites NIST SP 800-92 and Snodgrass et al. as supporting references [24, 37, 54]. The discrepancy-based approach used in this work therefore does not introduce the general concept of detecting log integrity violations. Instead, it applies an established detection principle to the specific domain of federated identity management and extends it with application-level observations from the Shibboleth IdP.

3.4 Derivation of the Rules

The derivation of the detection rules is based on three complementary sources: the FIM survey by Simpson and Groß, the OASIS SAML Security and Privacy Considerations, and the STRIDE threat classification scheme. The FIM survey identifies 11 vulnerabilities and 14 attack classes based on a systematic literature review [52]. The vulnerabilities are grouped into cross-protocol issues where the same weakness is exploited by the same attack class, results in the same CIA consequence, and is documented for at least two

different protocols. The survey identifies 14 such issues. Six of them concern SAML (issues 5, 7, 8, 9, 10 and 14 in the numbering of the survey), while one directly concerns Shibboleth (issue 6). The survey therefore provides the primary basis for identifying attack scenarios that are relevant to federated identity management.

While the survey describes vulnerabilities that have been documented in the literature, the OASIS SAML Security and Privacy Considerations provides security recommendations for SAML deployments by specifying security requirements and suggested countermeasures [21]. This allows the detection rules to be derived not only from observed or documented attacks, but also from the security properties that a SAML deployment is expected to enforce.

A third perspective is provided by STRIDE, a threat classification scheme that categorizes threats according to their mode of operation [51]. It distinguishes between spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege. Graves applies this framework to the analysis of threats against Shibboleth [18]. In this work, STRIDE is used to characterise the security impact of the attack scenarios identified by the other two sources.

The three sources therefore serve different purposes in the derivation process. The FIM survey identifies documented vulnerabilities and attack classes, OASIS specifies the security requirements that a SAML deployment should satisfy, and STRIDE provides a classification of the resulting security impact. Where the evidence from the survey and the requirements specified by OASIS overlap, the basis for a detection rule is particularly strong. Replay attacks provide an example of such an overlap, as they are documented in the literature and explicitly addressed by the SAML security considerations.

4 System Design

This chapter describes the design of the proposed monitoring system. It first derives the requirements that follow from the research questions and the preceding chapter, and then presents the system along the path that data takes through it. First, the data aggregation inside the IdP. Then, the transport of the collected data across a trust boundary. Finally, storage, correlation and rule-based detection.

The emphasis lies on the collection layer. The remaining layers follow established practice, whereas the question of which data can be obtained from within an IdP, and by which mechanisms, is specific to this work. Design decisions are stated together with the alternatives that were considered and rejected.

4.1 Requirements for the System

The requirements for the monitoring system were derived from the research and analysis of the related work. Section 3.3 showed that AppSensor's detection model provides no mechanism for comparing an observed event against a configuration declared by a third party. Since an IdP authenticates on behalf of other entities, such a comparison requires at least two independent sources. One source describing the intended configuration and another describing what actually occurred.

The classification in Section 3.2 placed this work at the *Monitor and Detect* level, with *Plan and Prevent* deliberately excluded. The system must therefore report detected security-relevant events without automatically intervening.

As mentioned in Section 3.2, Hall et al. identify the integration of detection points into the application as a central difficulty [19]. The system is therefore implemented as a plugin that uses an extension point provided by Shibboleth, allowing the monitoring functionality to be added without modifying the underlying IdP implementation.

Section 3.3 also showed that log sources stored on the same system share the same attack surface. Records generated by the interceptor are therefore transmitted to a separate system rather than stored locally. This requires the transport to be authenticated and sensitive data to be pseudonymised at the source.

The rules presented in Section 3.4, derived from the FIM survey by Simpson and Groß, the OASIS SAML Security and Privacy Considerations, and the STRIDE threat classification scheme, provide the actual detection logic for identifying unusual authentication behaviour and security-relevant conditions within the federated identity environment.

These considerations result in five requirements:

1. Two independent sources: the declared configuration and the observed event.
2. Detection and reporting without automated intervention.
3. Realisation as a plugin, without modification of the IdP.
4. Storage outside the monitored system, with authenticated transport and pseudonymisation at the source.

5. Detection rules derived from documented attack classes and security recommendations.

4.2 Overview

The proposed system comprises four layers. The data collection layer gathers security-relevant information from within the Shibboleth IdP, drawing on three sources: the audit log written by the IdP, its static configuration and metadata files, and two custom interceptors that are attached at two points of the profile processing and execute inside the IdP itself. Because the monitoring backend resides outside the trust boundary of the IdP, sensitive attributes such as usernames and IP addresses are pseudonymised before transmission. The transport layer then transfers the collected records to the backend application over an authenticated channel. Within the backend, incoming records are persisted and periodically correlated, so that observations originating from different sources but describing the same authentication event can be linked. The detection layer evaluates the resulting data against the rule catalogue in order to identify configuration weaknesses and anomalies in the authentication process. Finally, the presentation layer makes the detected findings accessible to the operator through a frontend web application. Figure 4.1 illustrates the UML deployment diagram of the proposed system.

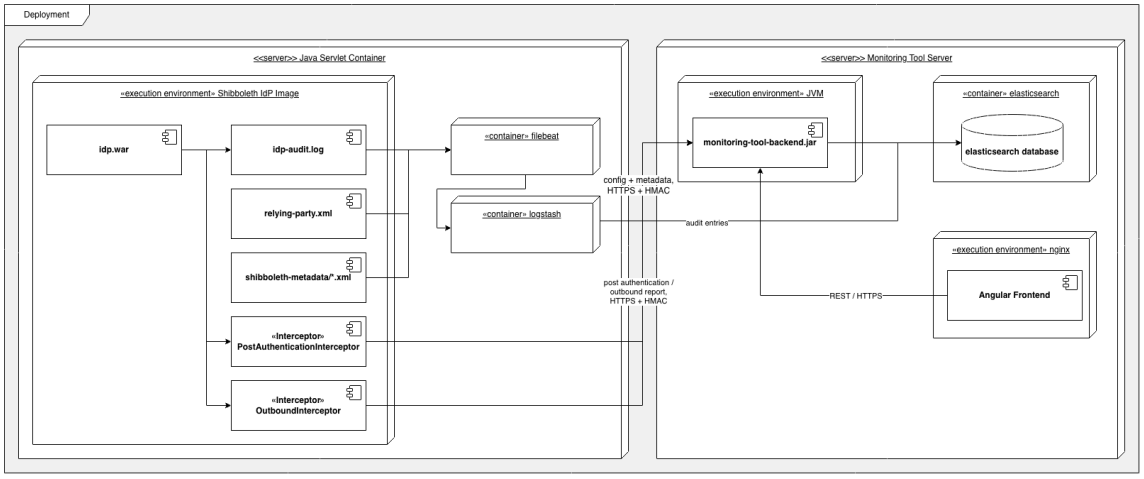


Figure 4.1: *UML Deployment Diagram of the Proposed Monitoring System*

Two terms recur throughout this work. An audit entry describes a record that the IdP writes to its audit log and that reaches the monitoring tool through the log shipping stack. A report represents a record that an interceptor produces within the IdP and transmits directly to the monitoring tool. Since two interceptors are used, their reports are called the post authentication report and the outbound report where they have to be distinguished. A report and an audit entry may describe the same authentication event, which makes the correlation described in Section 4.5.1 possible.

4.3 Data Aggregation Layer

The design of the collection layer begins with a broader question than the choice of individual data sources. Through which kinds of channel does an application disclose information about itself at all? Applied to the Shibboleth IdP, five such channels can be identified. The application writes log files, it reads static configuration and metadata, it maintains runtime state that is accessible through defined extension points, it exposes status endpoints, and it publishes metrics via Java Management Extensions.

Not all of these are suitable for the purpose of this work. The system described here associates every finding with an individual authentication event, since a security relevant condition is only meaningful in the context of the transaction in which it occurred. A channel that cannot be related to a single login can therefore not contribute to correlation or to the generation of an alert.

Two of the five channels fail this criterion. Status endpoints report the state of the deployment as a whole, such as its version, uptime, configured metadata providers or the status of the underlying services. They therefore address availability rather than the security of individual transactions. JMX metrics are aggregates by construction. A counter reports that a number of logins occurred, not which login was affected. An alert derived from either source could not be attributed to a transaction.

The remaining three channels form the collection layer of this work. These are the audit log, the configuration files, and the runtime state accessible through interceptors. They are described in the following sections.

4.3.1 The Audit Logs

All logging within the Shibboleth IdP is performed through the abstract logging API SLF4J, with Logback as the default implementation. By default, three classes of log files are produced. The diagnostic logs, *idp-process.log* and *idp-warn.log*, contain messages generated during the normal operation of the software. The general audit log, *idp-audit.log*, records the results of authentication transactions for each SP. Finally, the consent audit log, *idp-consent-audit.log*, contains audit records of user decisions regarding data privacy and terms of use [5].

The amount of information recorded in the diagnostic logs depends on the configured logging level, which determines the severity of messages that are written. The *idp-warn.log* file is a filtered view of *idp-process.log* and contains only messages logged at the `WARN` level or higher. These files are written for a human reader. A line consists of a timestamp, a severity, the originating class and a free-form message whose content depends on the code location that produced it, so that lines of different origin share no common structure.

The audit log differs from the diagnostic logs in that its structure is defined rather than free-form. It records one entry per completed transaction, written once the IdP has finished processing the request, with a fixed sequence of fields that allows every line to be decomposed in the same way. This has two consequences for the information it can provide. On the one hand, the entry reflects the final outcome of a transaction, including properties that are only determined at the end of processing, such as the encryption applied to the

outgoing assertion. On the other hand, events that never result in a completed transaction produce no entry. Failed password validations in particular are not recorded in the audit log by default, since no transaction is concluded for them.

As mentioned in Section 2.3.3, the fields that are written to the audit log are not fixed. The audit format is configured per profile in *conf/audit.xml*, where each field is represented by a formatting token, and the set of available tokens can be extended within the IdP [4]. Which information a given deployment records is therefore determined by its operator and may differ between installations.

This property has direct implications for the monitoring system developed in this work. The system cannot assume the presence of a fixed set of audit fields, since the information required by a particular detection rule may simply not be recorded. The detection logic must therefore take the configured audit format into account and determine whether the information a given rule depends on is available at all. The same constraint applies to the correlation between audit entries and interceptor records. Rather than relying on a fixed set of fields, the correlation must adapt to the information that a given deployment provides, which requires a strategy capable of selecting an appropriate matching approach depending on the fields available for a given audit event.

The audit format of the deployment was extended by the identifier and the issue time of the response and by the identifier of the request that it answers. These tokens make the response level fields that the outbound interceptor reads comparable with the audit entry, as described in Section 5.1.

4.3.2 Metadata and Relying Party File

In a Shibboleth IdP, SAML metadata describes the entities that participate in a federation and provides the information required for establishing communication between them. Metadata is represented in XML and contains information about the roles, endpoints, supported protocols, and cryptographic keys of the participating entities [6].

A SAML entity can support one or more roles, which are represented by role descriptors within the metadata. For SPs, the relevant role is represented by the `md:SPSSODescriptor` element. This descriptor contains information required by the IdP to communicate with the SP, including the supported SAML protocol families specified by the `protocol SupportEnumeration` attribute, cryptographic keys, supported name identifier formats, and the endpoints provided by the SP [8]. A central component of an SP's metadata is the `md:AssertionConsumerService` element. It specifies an endpoint to which the IdP can send SAML responses and assertions on behalf of the SP. By explicitly enumerating these endpoints in the metadata, the IdP can determine the locations to which SAML messages may be sent [8].

The Shibboleth IdP consumes metadata through configured metadata providers. These providers define the sources from which SAML metadata is obtained and are configured using `<MetadataProvider>` elements in *conf/metadata-providers.xml* [7]. The metadata therefore forms an essential part of the IdP configuration, as it determines which federation entities are known to the IdP and which endpoints, protocols, and cryptographic information are associated with them.

Alongside the runtime information recorded in the audit log, the IdP holds a second class of information that describes not what happened, but what is supposed to happen. This information is contained in the configuration files of the deployment, most notably in *relying-party.xml*.

This file specifies which functional features the IdP supports and allows these settings to be customised based on the identity of a relying party. It determines which profiles are enabled for a given partner and allows security configurations to be attached at various levels. These include whether assertions and other content are encrypted, at which layer signing is performed, and which cryptographic algorithms are used [11].

Of particular relevance to this work is that these settings are not global. A deployment defines default behaviour that applies to all partners, and may then override this behaviour for individual relying parties or groups of them. A single IdP therefore enforces a set of independent, per partner policies rather than one uniform policy, and the security properties of a given transaction depend on which policy applies to the SP involved.

Two settings are relevant in the context of this work. The first concerns encryption. A deployment may disable the encryption of assertions for a particular partner, for instance where an SP does not support it, which means that assertions for that partner are transmitted in plain text. The second concerns authentication. A deployment may specify which authentication flows are permitted for a partner, and thereby require multi factor authentication for some SP while accepting password based authentication for others.

Both settings describe a target state. Whether a given login actually satisfied it is recorded in the audit log, not in the configuration. Neither source is sufficient on its own. A comparison between them is only possible if both are available to the monitoring system, which is the reason why the configuration is treated as an independent data source in this work.

Finally, *relying-party.xml* is also the file in which interceptor flows are enabled for a profile. The extension point used by the collection mechanism described in Section 4.3.3 is therefore configured in the same place as the security settings it observes.

4.3.3 Interceptor

As mentioned Section 2.3.1 Shibboleth provides three interception points within its profile processing: inbound message processing, post-authentication processing, and outbound message processing. Interceptors are implemented as Spring Web Flow subflows that are executed at these predefined points without requiring modifications to the underlying profile flow [9, 10].

Inbound interceptors are executed during the processing of an incoming request. At this point, the inbound `MessageContext` tree is generally populated and, where applicable, contains a `RelyingPartyContext` as well as contexts required for operations such as transport or message authentication [9].

Consequently, an inbound interceptor can observe information associated with the received message and the context in which it is being processed. This makes the interception point particularly suitable for security checks on the incoming request. Shibboleth itself uses inbound interceptors for various security policy checks. An interceptor may also

modify the context tree, although such modifications directly influence subsequent profile processing [9].

For the monitoring system developed in this work, the inbound interception point is therefore relevant for observing properties of the incoming SAML message and its processing context before the main profile execution continues.

The post-authentication interception point is executed after authentication and attribute lookup have taken place, but before the profile constructs the outgoing message. Unlike the inbound and outbound hooks, this interception point is only available for a subset of profiles, particularly profiles that perform user authentication [9].

The interceptor therefore has access to a later state of the processing context. In particular, information related to the authenticated identity and resolved attributes is available at this stage. The interceptor can also modify the context before the outgoing response is constructed, including changes to the authenticated identity or attributes [9].

This makes the post authentication interception point particularly suitable for the monitoring system developed in this work, as it provides access to authentication related information that is not yet available at the inbound stage, and it does so before the profile constructs the outgoing message.

Outbound interceptors run shortly before the final stage of profile processing that produces the response to the client. Their input therefore represents the state produced by the preceding profile execution and typically contains a populated outbound `MessageContext`. The message may not yet be in its final form, since operations such as signing can still occur afterwards [9].

The outbound interception point consequently provides access to the information that is about to be returned to the relying party. Interceptors can inspect or modify the outbound context and may also terminate processing [9]. For the monitoring system, this interception point is useful for observing the result of the authentication workflow and properties of the response that is about to be issued by the IdP.

The monitoring system uses two interception points, the post authentication point and the outbound point. An inbound interceptor would see the incoming message but not the outcome of authentication, since that has not yet taken place. The post authentication point is the earliest point at which both the incoming message and the authentication result are available in the same context, and it serves as the main collection point. The outbound point still has access to both, since its input is the entire state of the tree that the profile execution has produced [9], and it additionally holds the response that is about to be issued. At the post authentication point, in contrast, the outgoing response does not exist yet. The outbound interceptor therefore complements the main collection point with the fields that only the response carries. The outbound point is also reached by responses that the IdP issues without authenticating the user, for instance an error response for an SP whose configuration cannot be fulfilled, which the post authentication point never sees.

What remains outside the view of the post authentication point is the outgoing response, including the response identifier, the final status of the transaction and the encryption actually applied. The outbound interceptor makes these properties observable, with one limit. When the assertion is encrypted, the encryption has already taken place when the

outbound hook runs, so that the assertion identifier, the authentication context and the name identifier are no longer readable, and the IdP cannot read them afterwards, since only the SP holds the private key. Reading them before the encryption would require an intervention in the profile flow itself, which the third requirement from Section 4.1 excludes. For encrypted assertions only the response level fields and the encryption state are therefore available to the interceptors, whereas the audit log records all of them, since the IdP extracts them at an earlier point of the processing sequence. The complementarity of the sources described in Section 4.3.1 therefore follows from their position in the processing sequence rather than from a deliberate division of responsibilities.

4.4 Transport Layer and Trust Boundary

After the data has been aggregated on the Shibboleth IdP side, it is transferred to the monitoring tool for further processing and storage. As stated in Section 4.2, this tool is deliberately deployed outside the trust boundary of the IdP. The separation avoids placing the monitoring system under the same security assumptions and access controls as the system it monitors.

If the monitoring functionality were deployed entirely within the IdP, an attacker who compromises the IdP could potentially compromise or manipulate the monitoring functionality at the same time. An attacker would then be in a position to influence the recording of their own activity, since collection, storage and analysis would all fall under the same control.

By transferring the collected information to a separate monitoring component, the observation and analysis functions are separated from the monitored application. This creates an additional security boundary and reduces the dependency of the monitoring system on the integrity of the IdP itself. The external component therefore does not exist solely for scalability or architectural convenience. What the separation protects is the data that has already been transmitted. An attacker who gains control of the IdP can influence which records are produced from that point onward, but not those that have already left the system. The independence is thus temporal rather than absolute.

This design decision has consequences. The collected data leaves the security domain in which the original authentication data is processed and becomes accessible to a separate component with a different trust relationship. Therefore, transmitting original sensitive user information, such as usernames and client IP addresses, would expose more personal information than is necessary for the intended processing purpose. A comparable concern is described by CWE-532 for log files, where the inclusion of sensitive user data provides attackers with an additional and less protected path to obtaining that information [31]. The same reasoning applies to data transferred to a component outside the original security domain.

To reduce this exposure, sensitive identifiers are pseudonymised before they leave the IdP trust boundary. The monitoring functionality does not require the original identity of a user to perform its security analysis. It primarily requires the ability to distinguish and correlate events belonging to the same subject. The monitoring system can therefore

preserve the ability to correlate multiple events belonging to the same subject without transmitting the original identifier. Pseudonymisation reduces the risk associated with unauthorised access to the monitoring system, while still allowing the required security analysis to be performed.

To enable this correlation across the different data sources, all components that generate monitoring data must apply the same pseudonymisation algorithm and the same secret salt. Otherwise, identical identifiers would result in different pseudonyms and events originating from different sources could no longer be reliably correlated. The secret salt is therefore treated as a shared cryptographic parameter between the data collection components and must be protected accordingly.

The deployment of the monitoring tool outside the IdP trust boundary introduces an additional security concern. The endpoints of the monitoring tool accept data from collection components that reside outside its own trust boundary. Consequently, the monitoring tool cannot assume that every received message originates from an authorised data collection component.

Without message authentication, an attacker could submit forged or manipulated records to the exposed endpoints. Injected events would not only produce false findings but could also interfere with the correlation of genuine ones, since a fabricated record may be matched against a real audit entry.

The transmitted data must therefore be authenticated before the monitoring tool accepts it. In this work, this is achieved using a Hash-based Message Authentication Code (HMAC) [25]. The collection components and the monitoring tool share a secret key that is used to generate and verify a message authentication code for each transmitted message.

The final design decision concerning the transport layer is whether the collection component should wait for the monitoring tool to confirm receiving the data. The interceptors executes within the authentication flow of the IdP. Any time spent transmitting a record is therefore time added to the login of the user who triggered it. Waiting for a response would make the duration of every authentication dependent on the availability and response time of a component outside the IdP, and a monitoring tool that is slow or temporarily unreachable would delay the service it observes.

The records are consequently transmitted asynchronously. Each interceptor dispatches its record and continues processing without awaiting a response. This has a cost. If a transmission fails, the collection component does not learn of it, and the record is lost without any indication at the point where it originated. The design therefore prioritises non interference with the monitored service over a guarantee of delivery. For a monitoring system that observes a central authentication service, this trade off appears justified. A monitoring component that degrades the availability of the service it observes would undermine its own purpose. Hall et al. decouple analysis for a similar reason: responses are retrieved separately rather than returned in reply to the submitted event [19].

4.5 Correlation and Detection Layer

This section explains how the monitoring tool processes the collected data. Records originating from different sources are first linked to the authentication event they describe, then evaluated against the detection rules, and finally classified according to the impact of the condition they report.

4.5.1 Correlation

When the collected data arrives at the monitoring tool, it must first be stored for subsequent processing. The different data sources transmit their information through separate endpoints and at different times, which means that information belonging to the same authentication event does not necessarily arrive together. The corresponding records can therefore not always be correlated upon arrival. Instead, they are stored until the required information from the other source becomes available.

This introduces the first challenge for the correlation process. Data collected by the post authentication interceptor within the Shibboleth IdP is transmitted immediately after the collection process has completed, so the corresponding post authentication report reaches the monitoring tool shortly after the authentication event has been processed. The audit entry follows a different path. Audit logs are collected by log shippers that operate periodically, and the corresponding entry therefore reaches the monitoring tool later than the post authentication report. This temporal offset is the reason why the two records cannot be matched by a simple database join at the time of arrival.

A further consequence follows from this. A report may arrive at a point when its counterpart does not yet exist, which means that an unsuccessful correlation attempt does not necessarily indicate that no counterpart will ever arrive. The correlation process must therefore retry over a defined period and, once that period has elapsed, treat the record as permanently uncorrelated. Without this distinction, a record still awaiting its counterpart would be indistinguishable from one that will never receive it, and the proportion of successfully correlated records could not be determined.

The second challenge arises from the dynamic nature of the audit log discussed in Section 4.3.1. Since the audit format is determined by the configuration of the deployment, the information available in an audit record is not fixed, and the correlation must adapt to the fields that are actually present.

This creates an asymmetric correlation problem. The data collected by the post authentication interceptor is static by design and always contains the same set of fields. The audit data, in contrast, is dynamic and may lack information required by a particular correlation strategy. The static post authentication report, which arrives first, therefore has to be matched against a dynamic audit record that arrives later.

Depending on the fields available in the audit record, different correlation strategies are applicable, and these differ in how reliably they identify the corresponding record. A strategy based on a unique identifier that both sources share establishes the relationship unambiguously, whereas a strategy based on attributes that several transactions may have in common can only approximate it. The correlation process consequently applies the

available strategies in a defined priority order, using the strongest applicable strategy first and weaker ones only where the required fields are absent. The strategy used for a given record is recorded alongside the result, since it determines how reliable the resulting correlation is. The precision achieved by each strategy is examined in Chapter 6.

4.5.2 Detection

Once the data has been collected and stored, it is evaluated against a catalogue of detection rules, where possible. With two exceptions described below, a rule receives a single object as input and either returns a finding or nothing. The conditions are stated explicitly rather than learned from data, which means that every finding can be traced back to the condition that produced it and, through the derivation in Section 3.4, to the documented weakness or requirement it is based on.

The rules do not all operate on the same kind of input. Four input types occur in this system: login events derived from the audit log, the configuration of an SP, its metadata, and the post authentication reports produced by the interceptor. These differ not only in structure but also in the rhythm at which they appear. Login events occur continuously during operation, whereas a configuration or a metadata document changes rarely and may remain unchanged for long periods of time.

Evaluating all rules against all data whenever any of it changes would therefore be inefficient and, in the case of configuration data, largely redundant. The detection logic is consequently divided into four engines, each responsible for one input type. An engine evaluates the rules that apply to its input and can be scheduled according to the frequency at which that input actually changes.

A further consideration follows from the dynamic audit format described in Section 4.3.1. A rule may depend on a field that a given deployment does not record, in which case the rule cannot reach a valid conclusion. Evaluating it regardless would produce a result that appears definite but rests on absent information, and a rule that cannot observe a condition must not report its absence as a finding. Each rule therefore declares which fields it requires, and the engine verifies their availability before evaluating the rule. Where the required information is not recorded, the rule withdraws rather than returning a result. Which rules are active in a given deployment is thus determined by what that deployment logs.

Which conditions the rules examine is a separate question from how they are evaluated. Thirteen of the fifteen rules can be traced to documented sources. Ten of them were derived from the three frameworks introduced in Section 3.4, namely the survey by Simpson and Groß, the OASIS security considerations and the STRIDE classification. `FAILED_LOGIN` reports an indicator rather than an attack and follows the OWASP guidance on security logging. The two rules that compare independently produced records follow the log integrity argument of Section 3.3. The remaining two rules cannot be traced and are discussed after Table 4.1, which shows the resulting assignment for each rule.

Two observations follow from this assignment. Lack of binding appears in three of the six SAML related entries and is therefore the most frequently documented weakness of the protocol in the survey. Both replay rules address it directly. Furthermore, several rules

Table 4.1: *Assignment of the 15 Detection Rules to Documented Vulnerabilities and Attack Classes*

Rule	Vulnerability	Attack Class	STRIDE	Source
WEAK_ENCRYPTION	Unencrypted Communications	Message Modification	I, T	[15, 21, 52]
UNENCRYPTED_ ASSERTION	Unencrypted Communications	Message Modification	I, T	[18, 52]
WEAK_ CONFIGURATION	Unencrypted Communications	Message Modification	I, T	[52]
MISSING_MFA	Weak User Credentials	Enables Brute Force	S, E	[52]
MFA_BYPASS	Weak User Credentials	Enables Brute Force	S, E	[52]
BRUTE_FORCE	Weak User Credentials	Brute Force	S, E	[52]
FAILED_LOGIN	Weak User Credentials	Indicator, not an attack	S	[36]
ASSERTION_REPLAY	Lack of Binding	Replay Attack	S, E	[21, 52]
REQUEST_REPLAY	Lack of Binding	Replay Attack	S, E	[21, 52]
SIGNATURE_ WRAPPING_ SUSPECTED	Message Formatting	Message Modification	S, T, E	[18, 56]
CERTIFICATE_ EXPIRED	Security recommendations	None	S	[21]
CORRELATION_ DISCREPANCY	Log integrity	Log tampering	T, R	[30, 36, 37]
FIELD_MISMATCH	Log integrity	Log tampering	T, R	[30, 36, 37]
SUSPICIOUS_USER_ AGENT	None	None	—	—
BEHAVIOUR_ANOMALY	None	Detection method	S	—

form pairs in which the same weakness is captured twice, once as a configuration state and once as an event that has occurred. MISSING_MFA examines whether an SP requires multi factor authentication at all, whereas MFA_BYPASS examines whether an actual login circumvented that requirement. This distinction is a direct consequence of the three source architecture described in Section 4.3: the configuration states what is intended, and the audit log records what took place.

Two of the rules do not examine a single record. They compare records that were produced independently of each other for the same event. The first, CORRELATION_DISCREPANCY, reports a post authentication report for which no audit entry arrived within the retry window of the correlation. The second, FIELD_MISMATCH, compares the fields of the outbound report with those of the audit entry that the post authentication report of the same login was correlated with, and reports every field in which they disagree. Both follow the detection point IE6 of AppSensor, the violation of security log integrity, and the argument of Section 3.3 that two independent records offer an alternative to protecting a log cryptographically [37]. In addition, input that is written to the audit log without

neutralisation can corrupt individual fields of an entry or split it into several lines, which the comparison with an independently produced record reveals as a field mismatch or, if the entry can no longer be correlated, as a discrepancy [30].

The comparison establishes that two records disagree but not which of them was altered, since an attacker who controls the application process can alter the report as well. It also cannot compare fields that only one of the records contains. For encrypted assertions this applies to the fields inside the assertion, which the outbound interceptor cannot read after the encryption and which only the audit log records. A field is compared only if the interceptor observed it and the audit format of the deployment records it, in line with the rule of the other detectors that a rule should not guess when the data it requires is absent.

Two rules cannot be assigned to any of the three frameworks. `SUSPICIOUS_USER_AGENT` identifies an indicator for which no documented relation to a specific attack class could be established. `BEHAVIOUR_ANOMALY` is a detection method rather than an attack class, since it may reveal several different attacks indirectly without corresponding to any one of them. Both rules are retained in the catalogue, and the absence of a derivation is stated here rather than omitted.

4.5.3 Severity Classification

Each alert detected by the detection layer described in Section 4.5 is assigned a severity level. The severity levels used in this work are based on the high-level risk categories proposed by Graves, which range from R1 to R5. Although Graves does not state an explicit ordering, he identifies the theft or abuse of the signing keys underlying R5 as the worst case. This work treats the categories as increasing in severity. The specific definitions of the risk categories are as follows [18]:

- **R1:** Abuse of system resources
- **R2:** Log on to a specific SP as any user
- **R3:** Log on to a specific SP as a specific user
- **R4:** Steal user passwords
- **R5:** Log on to any SP as any user

This classification describes the potential impact of an attack from an attacker's perspective. However, the rules developed in this work identify two different types of security conditions. Some rules detect vulnerabilities that can enable an attack, while others detect attacks or anomalous events that have already occurred. Severity classification is therefore performed in two steps. First, the identified risk determines the base severity level. Second, the classification is adjusted according to whether the rule identifies only a condition that enables an attack or evidence that the corresponding event has already occurred. A detected attack is therefore assigned a higher severity than a vulnerability that merely enables the attack.

This classification describes the potential impact of an attack from an attacker's perspective and serves as an orientation rather than as a formula. The severity is therefore assigned per rule, as listed in Table 4.2, and it is raised or lowered only where the impact of a rule depends on its context. The two multi factor rules are critical if the SP is on the

list of providers that must use multi factor authentication, and MFA_BYPASS also if the configured flow requires it in every case, and medium otherwise. WEAK_ENCRYPTION takes the severity from the algorithm policy, CERTIFICATE_EXPIRED from the expiry status of the certificate, and BEHAVIOUR_ANOMALY is rated high if at least two independent reasons apply and medium otherwise.

The discrepancy rule is classified as medium, since a missing counterpart can also result from operational causes such as a delayed log shipping chain. The mismatch rule is classified as high, since a contradiction between two records of the same event points either to a manipulation or to a defect in the collection, and both undermine the basis on which the other rules operate.

Table 4.2: *Severity assigned by each rule*

Severity	Rules
Critical	UNENCRYPTED_ASSERTION, WEAK_CONFIGURATION, ASSERTION_REPLAY, SIGNATURE_WRAPPING_SUSPECTED
High	BRUTE_FORCE, REQUEST_REPLAY, FIELD_MISMATCH
Medium	CORRELATION_DISCREPANCY
Low	FAILED_LOGIN, SUSPICIOUS_USER_AGENT
Context	MISSING_MFA, MFA_BYPASS, WEAK_ENCRYPTION, CERTIFICATE_EXPIRED, BEHAVIOUR_ANOMALY

4.6 Visualisation Layer

The primary purpose of the visualisation layer is to present the collected, correlated, and detected information to an operator through a user interface. The layer does not generate new information. Instead, it presents the data produced and aggregated by the preceding layers.

Accordingly, the interface provides visibility into detected findings but does not offer any functionality for modifying or controlling the IdP. This is a deliberate design decision rather than an omission and implements the second requirement described in Section 4.1, which follows from the passive monitoring principle adopted in this work.

Due to the pseudonymisation of user identifiers, the interface does not display plaintext identities. The monitoring system only retains pseudonymised identifiers and can therefore determine whether multiple events originate from the same user without revealing that user's actual identity. An operator who requires the original identity would have to obtain it from the IdP itself. Data protection is thus achieved through the architecture of the system rather than through access control alone.

The findings generated by the detection layer described in Section 4.5 are assigned severity levels according to the impact that a successful exploitation of the underlying weakness would have. The derivation of these levels is described in Section 4.5.3. They allow the operator to distinguish between findings of different importance and support the prioritisation of security relevant events.

In addition to the finding itself, the interface indicates which correlation strategy was used to link the underlying records. As described in Section 4.5, the available strategies differ in how reliably they establish the relationship between a post authentication report and an audit entry. Making this information visible allows an operator to judge how well supported a given finding is, rather than presenting all findings as equally certain.

4.7 Design Decisions and Rejected Alternatives

This section presents the main design decisions made for the monitoring tool and discusses the alternatives that were considered but rejected. Each decision involves a trade off, and the cost of the chosen option is stated alongside its rationale.

The first alternative was to extend the monitoring tool with the capability to actively intervene in ongoing attacks. In this design, the tool would provide the operator with the option to react to a detected finding by terminating the affected user session or by blocking the corresponding SP. This design was rejected because an incorrect response would not necessarily affect only the service concerned. Since the IdP provides authentication for multiple SPs, an erroneous intervention could disrupt several or even all connected services. The monitoring system is therefore intentionally designed as a passive system. The cost of this decision is that the system cannot prevent an attack that is in progress. It reports the condition it observes and leaves any reaction to the operator.

The second alternative was to incorporate cryptographic mechanisms for detecting log manipulation, following the approaches of Schneier and Kelsey and of Snodgrass et al. discussed in Section 3.3 [42, 54]. Such mechanisms can provide cryptographic evidence of modifications to stored log data. Their integration would, however, require changes to the existing logging and storage infrastructure and would introduce additional processing overhead, which Snodgrass et al. report as up to 16 percent [54].

The proposed system therefore does not provide cryptographic proof of log integrity. Instead, it detects potential inconsistencies by comparing audit log entries with information independently collected by the interceptors. This avoids modifying the existing logging infrastructure while still providing a second observation of the same authentication event. The cost is that an inconsistency indicates a discrepancy between two sources without establishing which of them was altered.

The third alternative was to use machine learning instead of explicitly defined detection rules. In such an approach, the system would learn normal behaviour statistically and identify deviations from it as anomalies. This approach was rejected because the detection rules developed in this work are explicitly derived from documented vulnerabilities, security requirements, and threat classifications. A rule based approach therefore provides direct traceability from a finding to the underlying security rationale, which is a central objective of the derivation described in Section 3.4. The cost of this decision is that the system detects only conditions that were anticipated when the rules were formulated. Attack patterns that do not correspond to any defined rule remain unnoticed, whereas a statistical approach could in principle identify previously unknown deviations.

A further decision concerns the number of interceptors. The system collects at two points, since the post authentication point was the basis of the collection and the outbound point was added to make the fields of the response observable. The outbound point turned out to see more events than the post authentication point. For an SP that was configured for encryption without a suitable key, the IdP answered with an error status before authentication began, as the empty subject and authentication fields of the audit entry show, and only the outbound interceptor produced a report. In principle, a single outbound interceptor could collect the data of both interceptors and halve the number of transmissions per login, since the contexts read by the post authentication interceptor are still part of the context tree at the outbound point. The assertion level fields of encrypted assertions would remain unavailable, as they are with the two interceptors used here. This was not examined beyond the request identifier, which the outbound interceptor already reads from the inbound message context.

Two further decisions have already been discussed in the context in which they arise. The choice of the interception points, described in Section 4.3.3, determines which information the collection mechanism can observe and which remains outside its view. The decision to transmit records asynchronously, described in Section 4.4, prioritises non interference with the monitored service over a guarantee of delivery.

5 Implementation

This chapter describes the implementation of the proposed monitoring tool and discusses the main challenges encountered during development. It follows the same structure as Chapter 4 by examining each system layer and describing its technical implementation.

5.1 Data Aggregation Layer

As described in Section 4.3, the data aggregation layer uses log shippers to collect and parse log entries and static configuration files, as well as two custom interceptors that operate within the Shibboleth IdP, the post authentication interceptor and the outbound interceptor, which are named after the hook at which they run. The log shipping stack consists of Filebeat for reading the source files and Logstash for preprocessing and transmitting the collected data. Both components run as separate containers on the same host as the IdP and access its log files and configuration through mounted directories. This placement is what allows them to share the pseudonymisation salt and the message authentication secret with the interceptor, as described in Section 5.2.

A key aspect of the implementation is the handling of audit log entries. Instead of parsing the audit log according to a fixed format, Logstash processes the audit fields as key-value pairs. This provides the technical basis for adapting to changes in the configured audit format and enables the monitoring system to handle the dynamic structure of the audit logs.

The monitoring tool handles these dynamic changes by maintaining a field registry. The registry periodically retrieves the most recent audit entries stored in the database and extracts the names of all fields that are currently present. Using only recent entries ensures that fields no longer written by the IdP are eventually removed from the registry. Both the detection rules and the correlation strategies query the registry before they are applied, which is how the withdrawal described in Section 4.5.2 is realised in practice.

The audit log is not the only log file read by the collection layer. As described in Section 4.3.1, authentication attempts that do not result in a completed transaction produce no entry there, which applies in particular to failed password validations. In the deployment used in this work, this authentication auditing is enabled and its records are routed to separate files, *idp-password-audit.log* and *idp-mfa-audit.log*, which record the outcome of the corresponding authentication flows. Filebeat is configured to read these files in addition to the general audit log, and the Logstash pipeline tags the resulting records with their source so that they remain distinguishable in the backend. Without this second source, the rules that evaluate failed authentication attempts would have no data to operate on.

A particular difficulty of this mechanism is that a missing field does not manifest itself as an error. If the audit format of a deployment does not include a field that a rule depends on, no exception is raised and no entry is written to any log. The information is simply absent, and a rule evaluating it would silently operate on incomplete data. The

registry therefore does not only serve adaptability, but also makes this absence explicit and detectable.

The static configuration files are collected by the same log shipping stack, but processed differently. Unlike the audit log, which is written continuously and line by line, *relying-party.xml* and the metadata documents change rarely and are read as a whole. Filebeat is therefore configured to aggregate the lines of such a file into a single event before transmitting it, and Logstash forwards the result without field level parsing. The number of lines combined in this way is bounded by configuration, which places an upper limit on the size of a document that can be transmitted as a unit. The interpretation of the XML structure takes place in the monitoring tool itself, where the relevant security settings are extracted and stored per SP. Filebeat monitors these files and transmits them only when their content has changed. The backend therefore does not receive the same configuration repeatedly, but only when a deployment has actually been modified, which is also what makes the detection rules operating on configuration meaningful at all.

Unlike the log shippers, the interceptor is a custom component developed as part of this work. It is registered as a Spring bean within the Shibboleth IdP and activated through the `postAuthenticationFlows` property of the corresponding profile configuration [10]. The post authentication interceptor therefore integrates into the IdP without any modification of its source code, which is how the third requirement of Section 4.1 is met.

The post authentication interceptor extends the `AbstractProfileAction` API of the Shibboleth IdP and operates on the `ProfileRequestContext`, which represents the current state of the profile execution [43, 46]. From this context it accesses the child contexts that carry the information relevant to the monitoring purpose. The `RelyingPartyContext` identifies the SP for which the authentication is performed, the `SubjectContext` provides the authenticated subject, and the `InboundMessageContext` gives access to the raw SAML request as it was received [45, 47, 50]. Further contexts, among them the `AuthenticationContext` and the `SessionContext`, supply information about the authentication flow that was used and the session state at the time of the request [44, 48].

The outbound interceptor is implemented as a second action class with its own Spring Web Flow subflow, which is registered next to that of the post authentication interceptor in the intercept configuration of the IdP. It is enabled for an SP in *relying-party.xml* through the *outboundInterceptorFlows* property of the profile configuration, in addition to the *postAuthenticationFlows* property of the post authentication interceptor. The flow contains a single state that calls the action and then proceeds, so that the processing of the profile is not influenced [10].

The action reads the outbound message context. If it does not contain a SAML 2 response, the action returns without effect. Otherwise it collects the status code, the identifier and the issue time of the response, the identifier of the request that the response answers, and whether the response contains encrypted assertions. If the assertion is not encrypted, it additionally reads the identifier and issue time of the assertion, the authentication instant, the session index, the authentication context class and the format and value of the name identifier, of which the value is pseudonymised before transmission in the same way as the subject and the address in the post authentication report. For encrypted assertions these fields are not collected, since the assertion is only available as

ciphertext at this point and the IdP holds no key to decrypt it. The destination attribute of the response is empty at this hook and is therefore not used. The identifier of the original request, which links the outbound report to the post authentication report, is read from the inbound message context, which is still available at this stage.

The outbound report is transmitted by the same mechanism as the post authentication report, including the signature over timestamp and payload and the switch between the asynchronous and the synchronous mode, but to a separate endpoint. The backend stores it in its own index with the request identifier as document identifier, and not as part of the post authentication report. The separation is necessary because the backend treats a second submission with the same identifier as a resubmission of the request and increments a counter that the replay rule evaluates. During development, registering the action of the post authentication interceptor at both hooks produced reports with a submission count of two and would have triggered that rule for every login.

The audit format of the deployment was extended by three tokens, the identifier of the response, the identifier of the request that it answers and the issue time of the response, so that the response level fields can be compared with the audit entry. The Logstash pipeline already mapped the corresponding tokens to fields.

The information obtained from these contexts is extracted, parsed, and combined into a single payload. Before this payload leaves the IdP, the sensitive identifiers it contains are pseudonymised, as required by the design described in Section 4.4.

The implementation of this step is complicated by the fact that it takes place in two different components. The interceptor pseudonymises the identifiers it collects within the Java application, whereas the audit fields are pseudonymised by Logstash during preprocessing. Both must produce the same value for the same identifier, since the correlation described in Section 4.5.1 relies on comparing hashed subjects and addresses across the two sources. A mismatch would not cause an error but would silently prevent records from being matched. Both components therefore apply the same hash function and share the same secret salt. The salt is provided through the deployment configuration rather than embedded in either component, so that it can be rotated without modifying the interceptor or the Logstash pipeline.

5.2 Transport Layer

Before the aggregated data is transmitted, the collecting component authenticates itself to the monitoring tool using an HMAC signature, as outlined in Section 4.4. Both Logstash and the interceptor hold the shared secret and sign the records they send, so that the backend can verify that a received message originates from an authorised collection component.

The verification is conditional on configuration. If no secret is configured on the sending side, the message is transmitted unsigned, and the backend enforces validation only where a secret has been configured there as well. This makes it possible to deploy the system without prior key distribution, for instance in a test environment, at the cost of accepting

unauthenticated records in that case. A production deployment is therefore expected to configure the secret on both sides.

The monitoring tool provides a separate endpoint for each type of collected data. A post authentication report, an outbound report, an audit entry and a configuration document differ in structure and in the processing they require, so that separating them at the interface avoids having to determine the type of a record before it can be handled.

As mentioned in Section 4.4, the interceptors send their payload asynchronously. Neither of them waits for a response from the monitoring tool, and each proceeds with the authentication flow immediately after dispatching its record. If a transmission fails, whether because the backend is unreachable, the request times out or the record is rejected, the interceptor receives no indication of it and the record is lost without a trace at its point of origin.

To quantify the cost of this design decision, the interceptors also support a synchronous mode. In this mode each of them waits for the backend to confirm receipt before the authentication flow continues, which makes the time spent on transmission visible in the overall duration of a login. The mode is not intended for productive use but serves as a reference point against which the asynchronous variant can be compared. The resulting measurements are presented in Section 6.4.

The synchronous mode applies to both interceptors and is selected by the same environment variable, so that a login in that mode contains two blocking transmissions. Where a secret is configured, both interceptor endpoints accept only requests that carry a valid signature.

5.3 Correlation and Detection Layer

This section describes the main technical components of the backend application, following the same structure as Section 4.5.

5.3.1 Storage

Elasticsearch serves as the primary data store of the backend application. The choice follows from the data collection layer described in Section 5.1, since Filebeat and Logstash write directly into Elasticsearch and no further conversion is required for the data they deliver.

The application uses separate indices for the different types of data it handles, namely audit entries, metadata, configuration data and the reports of the two interceptors. These indices differ in how they are created. Audit entries are written by Logstash into time based indices that are rotated daily, whereas post authentication reports and outbound reports are stored in a single index each, maintained by the application itself. Queries that span audit data therefore address an index pattern rather than a concrete index, which restricts the set of Elasticsearch operations available for them.

Elasticsearch does not provide relations between documents. The result of a correlation is therefore stored as a field on the post authentication report, which holds the identifier of the audit entry it was matched with. The strategy that produced the match is stored

alongside it, as described in Section 4.5.1, so that the basis of a correlation remains visible after the fact.

5.3.2 Correlation

This section describes how the two challenges of the correlation layer identified in Section 4.5.1 are addressed in the implementation, namely the temporal offset between the arrival of related records and the varying availability of fields in the audit log. The post authentication report is static and always contains the fields required for correlation. The constraint therefore lies entirely on the audit side. The audit logging documentation lists all fields that a deployment may record [4], of which four are usable for establishing a relationship between an audit entry and a post authentication report.

- **I**: the inbound message identifier of the SAML request
- **u**: the username, pseudonymised by Logstash
- **a**: the client IP address, pseudonymised by Logstash
- **T**: the timestamp of the event

The audit format provides further identifiers that are unique per transaction. The `InResponseTo` value of the response echoes the identifier of the inbound request, and a strategy built on it would establish the same one-to-one relationship as the inbound message identifier already does, with no difference in outcome. The assertion identifier and the outbound message identifier are unique as well, but they are created after the post-authentication hook has run, which executes before the outgoing message content is constructed [9], and are extracted at later points of the audit pipeline [4]. They are therefore not available to the interceptor at the post authentication hook. The outbound interceptor described in Section 5.1 reads the outbound message identifier and, for assertions that are not encrypted, the assertion identifier, and the audit format of the deployment records the outbound message identifier as well, so that these fields serve the comparison of the two records rather than the correlation. Which identifiers can be used for correlation is thus determined by the observation point rather than by the audit format alone. The remaining fields, among them the entity identifier of the SP, the authentication flow and the released attributes, are identical across many logins of the same partner and narrow the candidate set no further than the SP already does.

These fields differ in how precisely they identify a counterpart. The inbound message identifier is unique per request and therefore establishes the relationship unambiguously, since the same value appears in both records. The two pseudonymised identifiers narrow the set of candidates to those belonging to the same subject and origin, but several logins of the same user may share them. The timestamp alone can only indicate temporal proximity. In addition, all strategies restrict candidates to the SP recorded in both records, since an audit entry for a different partner can never be the counterpart of a given report. Where the inbound message identifier is not recorded, the relationship can therefore only be approximated. If none of the identifiers is available, the timestamp remains the sole criterion.

The three correlation strategies implement a common interface and are applied in a fixed order of precedence. Each strategy declares the audit fields it depends on and queries the field registry described in Section 5.1 before it is applied. Where a required field is not recorded by the deployment, the strategy withdraws and the next one in the chain is attempted.

The first strategy, `INBOUND_MESSAGE_ID`, matches the inbound message identifier of the post authentication report against the corresponding audit field. Since this value is unique per request, a match establishes the relationship without ambiguity. The second strategy, `HASH_AND_TIME_WINDOW`, narrows the candidates by SP, pseudonymised subject and pseudonymised client address, and additionally restricts them to a configurable time window around the event. The third strategy, `TIME_WINDOW`, retains only the SP and the time window, and is therefore used where neither the message identifier nor the pseudonymised identifiers are recorded.

The pseudonymised subject appears under two names, since the post authentication report stores it as *subjectHash* whereas the audit entry records it as *usernameHash*. Both hold the same value, derived from the same identifier with the same salt, and the distinction is one of naming within the respective data source rather than of content.

Both time window strategies may return more than one candidate. In this case the entry with the smallest temporal distance to the post authentication report is selected. The strategy that produced a match is stored alongside the result, so that the reliability of a given correlation remains apparent afterwards.

The timestamp used for these windows requires particular attention. The field that Elasticsearch assigns to an audit document by default reflects the point at which Filebeat read the log line, not the point at which the authentication took place. Since Filebeat processes log lines in batches, this value is shared by all entries of a batch and does not preserve their order. The Logstash pipeline therefore derives a dedicated field from the event timestamp recorded by the IdP, and both time window strategies operate on this field. The effect of this distinction on correlation precision is examined in Chapter 6.

The temporal offset described in Section 4.5.1 is addressed by repeated attempts. A post authentication report that could not be correlated on arrival is retried periodically, with a minimum age before the first attempt to account for the indexing delay of the log shipping stack. After a defined retry window has elapsed without a match, a separate scheduled sweep marks the report as permanently uncorrelated. This marking is what allows an uncorrelated report to be distinguished from one that is still awaiting its counterpart, and thus what makes the proportion of successfully correlated reports determinable at all.

Two mechanisms prevent an audit entry from being claimed twice. Within a single retry run, entries that have already been assigned are excluded from the candidate set of subsequent reports. Across runs, the application verifies against the stored data whether an audit entry has already been referenced by another report before accepting a match.

5.3.3 Detection

The detection logic is divided into four engines, following the separation by input type described in Section 4.5.2. The rule engine evaluates login events derived from audit

entries, the configuration rule engine evaluates the security settings of a SP, the metadata rule engine evaluates its metadata, and the interceptor rule engine evaluates the post authentication reports produced by the interceptor.

All rules implement a common interface. A rule receives a single object of the type its engine is responsible for and returns either an alert or nothing. Each rule additionally declares which fields it depends on, and the engine queries the field registry described in Section 5.1 before evaluating it. Where a required field is not recorded by the deployment, the rule is skipped rather than evaluated on incomplete data.

The engines are executed by a scheduler at intervals that reflect the rhythm of their input. Login events and post authentication reports arrive continuously and are therefore evaluated frequently, whereas configuration and metadata change rarely and are checked at longer intervals. To avoid evaluating the same login events repeatedly, the application maintains a marker that records the point up to which audit entries have already been processed. Each run evaluates only entries newer than this marker, and the marker is advanced once the run has completed. A grace period is applied before an entry is considered, so that records still being indexed by the log shipping stack are not skipped.

A further mechanism concerns repeated findings. A weakness in a configuration persists until the operator changes it, and evaluating the same configuration every few minutes would otherwise produce a continuous stream of identical alerts. The application therefore stores at most one alert of a given type per subject and day, so that a persistent condition is reported once rather than repeatedly.

The rules differ in the kind of condition they examine. Three examples illustrate this range. `WEAK_ENCRYPTION` compares the encryption algorithm recorded for a login against a list of algorithms considered insecure, which is maintained in the application and can be adjusted without changing the rule itself. `BRUTE_FORCE` does not evaluate a single event in isolation but counts failed authentication attempts of the same subject within a defined time window, which requires a query across previously stored events. `SIGNATURE_WRAPPING_SUSPECTED` operates on the raw SAML request contained in a post authentication report and examines its structure for duplicate identifiers and for signature elements referencing an identifier that occurs more than once.

The remaining rules follow one of these patterns or compare an observed event against the configuration declared for the SP involved, as described for `MFA_BYPASS` in Section 3.3. Their assignment to documented vulnerabilities and attack classes is given in Table 4.1.

A further difficulty arose with the comparison between configuration and runtime event. The authentication flows permitted for an SP are listed in *relying-party.xml* without a namespace prefix, whereas the audit log records the flow that was actually used with one. The same flow therefore appears as `MFA` in the configuration and as `authn/MFA` in the audit entry. A direct string comparison between the two never matches, and the rule would consequently never report a finding. The comparison is therefore performed token by token and accepts both notations. It is implemented in a shared comparison function that is used by all rules evaluating authentication flows. As with the two cases described in Section 5.1, the failure mode here would have been silent, since neither an exception nor a log entry would have indicated that the comparison was ineffective.

Two rules operate on pairs of records and are therefore not evaluated by the rule engines. Both are triggered by the correlation service. When a report is marked as permanently uncorrelated, either because the retry loop reaches the end of its window or because the periodic sweep finds it expired, the service raises a `CORRELATION_DISCREPANCY` alert of medium severity. Each report is expired exactly once, so that the alert needs no deduplication, which distinguishes it from the alerts of the rule engines, of which at most one per type is stored per day.

When a report is correlated with an audit entry, the service loads the outbound report with the same request identifier and passes it together with the audit entry to the consistency service. If no outbound report exists, the check is skipped. Otherwise the service compares the fields listed in Section 6.3.7. A field is compared only if the interceptor observed a value for it and the field registry shows that the audit format of the deployment records it. If the audit entry lacks a value for a field that the registry lists, the field counts as deviating, since a removed field is itself a sign of tampering. Timestamps are parsed and compared as instants, and the other values as strings after the abbreviations of the audit format have been mapped back to the notation of the report. The map mirrors the default map of the IdP and is a copy that is not synchronised with its configuration. On a deviation the service stores a `FIELD_MISMATCH` alert of high severity, whose details list each deviating field with the values of both records.

5.4 Visualisation Layer

The visualisation layer displays the collected, correlated, and detected data through an Angular based web application. A generic dashboard such as Kibana was not used, since the views required here are specific to the domain of the monitoring tool. They present findings together with the records from which they were derived and follow the relationships that the correlation layer establishes, neither of which can be expressed through generic index queries alone. The main pages implemented for the web application are described below.

The dashboard provides an overview of the most important data within a specified timeframe. It includes the total number of logins, failed logins, successful logins, and detected alerts. In addition, it provides a line chart showing the number of logins over the selected timeframe, an overview of the total number of logins per SP, and a detailed table of the individual login events.

The login overview for each SP also provides a detailed dialog containing the configuration and metadata defined for the respective SP in the Shibboleth IdP. The SP details further include an overview of any misconfigurations detected by the rule engines described in Section 5.3.3.

The second major page of the web application is the alert page. As its name suggests, this page displays the recent alerts detected by the monitoring tool. It includes a vertical bar chart showing the alerts detected during the current week, grouped by their respective severity. The page also provides a table of alerts ordered by severity.

Each row in the alert table is interactive and opens a detailed dialog containing information about the selected alert. This dialog also provides access to the audit entry and the post authentication report associated with the alert. Conversely, both the audit entry and the post authentication report dialogs list all alerts detected for the corresponding event. The relationship established by the correlation layer is therefore navigable in both directions, which allows an operator to examine a finding together with the records it was derived from rather than in isolation.

The audit entry view displays only the information that was available in the corresponding audit file of the Shibboleth IdP. Since subjects and client addresses are pseudonymised before leaving the IdP, as described in Section 5.1, the interface presents these values in their hashed form. An operator can therefore recognise that several events originate from the same subject without learning the identity of that subject.

Each alert can also be acknowledged by an operator and marked as a false positive. Acknowledged alerts and false positives are removed from the default alert view, which only displays active alerts.

The third major page displays the collected post authentication reports in a table. Each row contains the most relevant information from the corresponding post authentication report and indicates whether the report was successfully correlated with an audit entry. This makes the state described in Section 5.3.2 visible to the operator, including reports for which no counterpart was found within the retry window. Each row can be expanded to reveal additional details, including the raw SAML request, and a button allows the operator to open the corresponding alert in a detailed dialog.

6 Evaluation

This chapter evaluates the proposed monitoring system. It presents the test setup, evaluates the correlation and detection capabilities, measures the performance overhead introduced by the interceptors, and concludes with a discussion of the threats to validity.

6.1 Test Setup

To facilitate integration with the Java testing framework, the entire test environment runs in containers. This comprises the Shibboleth IdP, an LDAP directory serving as the user store, the SPs, and the backend application of the monitoring tool.

Two SPs are deployed, referred to as the weak SP and the clean SP in the following. The weak SP is used for the detection scenarios and is configured to exhibit the weaknesses that the rules are intended to identify. The clean SP is configured according to current recommendations and serves as a reference for the false positive rate, since any alert raised for a legitimate login against it is by definition a false positive.

The evaluation distinguishes three levels of testing. Unit tests examine individual components in isolation, with their dependencies replaced by mocks, so that a test result reflects the behaviour of that component alone. Integration tests run against a real Elasticsearch instance, which additionally covers the persistence layer, the index mappings and the queries that a mock cannot represent. End-to-end tests exercise the complete pipeline, from a login at the SP through the log shipping stack to the alert produced by the backend.

Logins are generated in two ways. An HTTP client harness performs the SAML login sequence directly, parsing the forms returned by the IdP and submitting them without rendering the pages. Selenium based tests drive an actual browser through the same sequence. The two differ in the interval between consecutive logins, since the harness proceeds as fast as the server responds whereas a browser adds the time required for page loading and form interaction. This difference is relevant for the correlation measurements in Section 6.2, where the temporal density of logins determines how reliably records can be matched.

The environment is not a production deployment. It comprises a single IdP, two SPs and a small number of test accounts, and all traffic is generated for the purpose of measurement. The implications of this are discussed in Section 6.5.

6.2 Correlation

Section 5.3.2 described the correlation mechanism, which links each post authentication report to the audit entry of the same login. It first tries `INBOUND_MESSAGE_ID` and falls back to two heuristic strategies where the identifier is not available. Both heuristics query the audit entries of the same SP within a time window around the report. `HASH_AND_TIME_WINDOW` additionally requires the same *usernameHash* and *ipHash*, whereas `TIME_`

WINDOW relies on the time alone. The entries returned by such a query are referred to as candidates in the following, and the strategy selects the candidate with the smallest temporal distance to the report. This section examines how often the selected candidate is the correct one (precision) and how many reports find a counterpart at all (coverage). The exact strategy serves as the reference for the heuristics, and the measurement is repeated on datasets that differ in the number of users and in the login generator.

6.2.1 Method and Datasets

Ground truth. To determine the precision of the heuristic strategies, a reference is required that indicates whether a given correlation was correct. The correlation via INBOUND_MESSAGE_ID establishes a one-to-one relationship, since the identifier is unique per request. Wherever this strategy produced a match, the correct counterpart is therefore known without ambiguity, which allows it to serve as a ground truth for the heuristic strategies. An analysis program in the test sources of the backend proceeds as follows:

- It collects all post authentication reports of a configurable time period that were correlated via INBOUND_MESSAGE_ID.
- For each report and each window size from ±1 to ±60 s, it executes the query of the heuristic strategy unchanged and selects the same candidate as the productive system would, namely the one with the smallest temporal distance.
- It compares the selected candidate with the known correct audit entry and classifies the outcome as a hit, an incorrect assignment or no match. Precision is the share of hits among the assigned reports, coverage the share of reports that were assigned at all.
- It records the offset between the reception of the report and the event time of its audit entry, and the distance to the nearest other audit entry of the same SP as a measure of the local login density.

Each report is evaluated on its own. The productive filter that excludes audit entries already assigned to another report is therefore omitted, since in retrospect it would exclude exactly the entry that the exact match has claimed.

Two corrections before the measurement. The ground truth requires the inbound message identifier in the audit log. It was missing from the audit format of the IdP, so that the exact strategy had never been active and all 3,690 correlations up to that point had been established by HASH_AND_TIME_WINDOW. After %I had been added to *conf/audit.xml*, the strategy activated itself during operation without a restart, as intended by the dynamic field adaptation of Section 5.1. The second correction concerns the time base of the heuristics, which initially compared against the ingest time assigned by Filebeat. Since Filebeat assigns one value to all entries of a batch, several logins shared the same time and could not be ordered. Logstash now derives the field *event_time* from the event time recorded by the IdP (%T), and all measurements below use it.

It records the offset between the reception of the report and the event time of its audit entry, and the distance to the nearest other audit entry of the same SP as a measure of the local login density.

Each report is evaluated on its own. The productive filter that excludes audit entries already assigned to another report is therefore omitted, since in retrospect it would exclude exactly the entry that the exact match has claimed.

Two corrections before the measurement. The ground truth requires the inbound message identifier in the audit log. It was missing from the audit format of the IdP, so that the exact strategy had never been active and all 3,690 correlations up to that point had been established by `HASH_AND_TIME_WINDOW`. After `%I` had been added to *conf/audit.xml*, the strategy activated itself during operation without a restart, as intended by the dynamic field adaptation of Section 5.1. The second correction concerns the time base of the heuristics, which initially compared against the ingest time assigned by Filebeat. Since Filebeat assigns one value to all entries of a batch, several logins shared the same time and could not be ordered. Logstash now derives the field *event_time* from the event time recorded by the IdP (`%T`), and all measurements below use it.

Datasets. Table 6.1 lists the datasets, each of which closes a gap left by the previous ones. With a single user, all candidates share subject, address and SP, so that only the temporal order separates them and both heuristics select from identical candidates. D1 therefore tests whether the event time is precise enough to order consecutive logins. D2 adds two further users, which makes the hash filter matter and tests `TIME_WINDOW` for the first time. Since the harness sends logins as fast as the server responds, D3 and D4 repeat both cases with a browser to check whether the timing of the harness is representative. In D4 each browser runs in its own container with its own address, which allows a reanalysis without *usernameHash* to isolate the contribution of *ipHash*. D5 covers the opposite case of several users sharing one address, as behind network address translation, by running the users as threads of one process. Coverage is determined on the performance series of Section 6.4, the largest dataset and the one recorded with the final configuration.

Table 6.1: *Datasets of the Correlation Measurements*

Dataset	Login generator	Users	Reports
D1	HTTP harness	1	203
D2	HTTP harness	3	600
D3	Selenium	1	202
D4	Selenium, one container per user	3	606
D5	HTTP harness, threads of one process	3	600
Performance series	HTTP harness	1	7,236

6.2.2 Defects Identified during the Evaluation

Before the evaluation, 3,690 out of 3,690 correlations had been established through the hashed identity and time window strategy, and not a single one through `INBOUND_MESSAGE_ID`. The figure comprises all post authentication reports that had accumulated in the test environment up to that point, across the development and benchmark runs performed before the correlation evaluation began. The cause was not located in the application but in the audit format of the IdP. The *shibboleth.AuditFormattingMap* in *conf/audit.xml* did not contain `%I`. The Logstash pipeline was already mapping *[shib][I]* to the corresponding field, but it was waiting for a value that the configuration never produced. The precondition of the strategy, namely the presence of the inbound message identifier in the field registry, therefore remained permanently unmet. After adding `%I` to the audit format, the registry recognised the field and the strategy activated itself during operation, without restarting the system. This is direct evidence of the dynamic field adaptation described in Section 5.1.

A second finding concerns the timestamp used by the time window strategies. These compared against the *timestamp* attribute, which reflects the ingest time assigned by Filebeat rather than the time of the authentication event. Since Filebeat processes log lines in batches, this value is shared by all entries of a batch. A measurement on benchmark traffic showed that 2,423 audit entries were distributed across only 448 distinct *timestamp* values, an average of 5.4 events per timestamp. Within such a group the temporal distance is identical for all candidates, so the chronological order of several logins could not be reconstructed.

This was resolved by introducing a dedicated field *event_time*, which Logstash derives from `%T`, the event time recorded by the IdP, using a date filter. Both time window strategies have operated on this field since. The *timestamp* attribute remains the basis for scheduler markers and the indexing grace period, where the ingest time is the factually correct reference.

6.2.3 Results of the Evaluation

With *event_time* as the basis, the temporal resolution improves to 1.0 events per timestamp. Each event therefore carries a distinct value, in contrast to the 5.4 events per timestamp observed with the ingest time. Table 6.2 and Table 6.3 show the precision of the heuristic strategy against the exact ground truth before and after this change. The two are based on separate runs, since the change to *event_time* required the data to be collected anew.

Both measurements were performed with a single test user against a single SP, which means that all candidate records share the same subject, the same address and the same partner. Cases in which these assumptions do not hold are examined in the following sections.

Table 6.4 shows the distribution of this offset. The predominantly negative offset is consistent with the position of the post authentication interceptor in the profile flow. It runs as a *postAuthenticationFlow* in the middle of the profile, whereas `%T` marks the

Table 6.2: *Precision of HASH_AND_TIME_WINDOW based on the Filebeat ingest time, single user via HTTP harness (n = 415)*

Window	Hits	Mismatch	No Match	Precision	Coverage	Candidates
±2 s	24	373	18	6.05%	95.66%	11.79
±5 s	27	384	4	6.57%	99.04%	28.90
±10 s	29	386	0	6.99%	100%	55.47
±30 s	29	386	0	6.99%	100%	138.48
±60 s	29	386	0	6.99%	100%	192.60

Table 6.3: *Precision of HASH_AND_TIME_WINDOW based on the IdP event time, single user via HTTP harness (n = 203)*

Window	Hits	Mismatch	No Match	Precision	Coverage	Candidates
±1 s	203	0	0	100%	100%	6.42
±2 s	203	0	0	100%	100%	12.36
±5 s	203	0	0	100%	100%	30.24
±10 s	203	0	0	100%	100%	58.12
±30 s	203	0	0	100%	100%	144.68
±60 s	203	0	0	100%	100%	197.60

completion of the transaction. The report therefore typically reaches the backend around 30 ms before the IdP writes the corresponding audit entry.

6.2.3.1 Precision of TIME_WINDOW under True Candidate Ambiguity

The measurements in Table 6.3 are based on a single test user. In that case the candidate sets of HASH_AND_TIME_WINDOW and TIME_WINDOW are identical, since a second *username-Hash* never occurs within the observation period. The weaker strategy is therefore never actually tested, only carried along. To address this, the same measurement methodology was applied to a dataset with several test users active in parallel.

The extended dataset comprises three users with 200 logins each, started simultaneously against the same SP, resulting in 600 reports in total. The ground truth was cleared before the run to prevent any mixing with the single user data. The results are summarised in Table 6.5. The measurement covers six window sizes, namely ±1, ±2, ±5, ±10, ±30 and ±60 seconds.

The precision of TIME_WINDOW is identical across all six measured window sizes at 65%. This follows directly from the selection rule, according to which the candidate with the smallest temporal distance is chosen. Once the correct candidate lies within the window, a more distant candidate admitted by a larger window can no longer displace it. Since the number of reports without a match is consistently zero, the ±1 s window already sufficed to include the correct candidate, only not always as the closest one.

Table 6.6 shows that the incorrect assignments of TIME_WINDOW are concentrated entirely among logins that lie less than 100 ms apart from the next login of the same SP. 521 of the

Table 6.4: *Temporal offset between post authentication report reception and IdP event time (n = 203)*

min	p50	p95	max	Span
−98 ms	−32 ms	−23 ms	+75 ms	173 ms

Table 6.5: *Precision of both strategies under three concurrent users via the HTTP harness (n = 600)*

Strategy	Precision (all windows 1–60 s)	Coverage	Candidates (±1 s)
`HASH_AND_TIME_WINDOW`	100%	100%	5.66
`TIME_WINDOW`	65%	100%	16.26

600 reports fall into this category. Above an interval of 100 ms the strategy is error-free within the measured dataset.

6.2.3.2 Precision under Real Browser Traffic

All measurements presented so far are based on the synthetic HTTP login harness. To examine whether its timing behaviour is representative of real login processes, the same measurement methodology was applied to a second dataset generated through Chrome-based logins using Testcontainers and Selenium instead of a raw HTTP client. The data source is 202 logins of a single test user, comprising 200 measured logins and the hard-coded warm-up of two. The results are summarised in Table 6.7, where the HTTP client column repeats the values of Table 6.3.

As Table 6.8 shows, all 202 logins lie more than one second apart from their nearest neighbour. The candidate mean at ±1 s is 1.00 instead of 6.42, which places the login density roughly six times lower than with the HTTP client harness. This is consistent with the additional time required for real browser interaction, comprising page loading, form handling and the overhead introduced by Selenium, compared to a script without waiting times. At this density `HASH_AND_TIME_WINDOW` and `TIME_WINDOW` yield identical results, since with an average of one candidate per window the additional hash filter has nothing left to exclude, for the same reason as in the single user case.

The offset between report reception and event time ranges from −102 to −17 ms (n = 202), which is narrower and consistently negative compared to the range reported in Table 6.4. With a single run of this size no reliable trend can be established, but there is equally no indication that real logins are temporally more irregular than the synthetic client.

6.2.3.3 Precision among Multiple Simultaneous Browser Users

What remains open is the combination of the two preceding measurements, namely several users and real browser traffic at the same time. The same measurement methodology was therefore applied with three test users running simultaneously against the same SP. The data source is 202 logins each, comprising 200 measured logins and the hard-coded

Table 6.6: *Precision of `TIME_WINDOW` by local login density, HTTP harness, window = 10 s*

Density Bucket	n	Hits	Mismatch	Precision
< 100 ms	521	311	210	59.69%
100–300 ms	65	65	0	100%
300 ms–1 s	14	14	0	100%
> 1 s	0	0	0	—

Table 6.7: *Single user, HTTP harness compared to real browser logins*

	HTTP client (n = 203)	Browser (n = 202)
Precision (both strategies)	100%	100%
Candidates at ±1 s	6.42	1.00
Candidates at ±60 s	197.60	42.61

warm-up of two, resulting in 606 reports in total. The results are summarised in Table 6.9, Table 6.10 and Table 6.11.

The precision of `TIME_WINDOW` within the critical sub-100 ms zone is nearly identical for real and synthetic traffic, at approximately 56 to 60 percent. The weakness measured in Section 6.2.3.1 is therefore not an artifact of the HTTP client timing, but a reproducible density effect. What differs is how frequently this zone is reached at all. The artificially dense stress test enters it in 87% of cases, whereas three genuinely concurrent browser users do so in only 23%. The lower overall precision of the synthetic dataset therefore describes the stress case correctly, but overstates how often it occurs under the observed user load.

6.2.3.4 The Independent Contribution of *ipHash*

It remained unclear whether *ipHash* actually contributed to the separation in the multi-user datasets, or whether *usernameHash* alone accounted for the measured precision. The initial assumption had been that all benchmark processes would share a common *ipHash*. A control query on the raw data refuted this for the browser path, since each `BrowserWebDriverContainer` started in parallel is assigned its own Docker bridge address. The three test users in Section 6.2.3.3 therefore had three distinct *ipHash* values rather than a shared one.

This could be verified against the existing ground truth without performing a new login run. `CorrelationPrecisionAnalysis` was extended by a third, purely analytical strategy that searches by SP, *ipHash* and time window while omitting *usernameHash*. This is not a productive strategy but a diagnostic variant introduced to isolate the contribution of *ipHash*. The data source is the same 606 reports as in Section 6.2.3.3, reanalysed with the additional strategy. Table 6.12 shows the result of this comparison.

The comparison across all six window sizes shows not merely a convergence but exact agreement down to the second decimal place, as Table 6.13 shows.

Table 6.8: *Local login density under real browser traffic, single user (n = 202)*

Density Bucket	n	Precision
< 100 ms	0	—
100–300 ms	0	—
300 ms–1 s	0	—
> 1 s	202	100%

Table 6.9: *Precision and coverage under three concurrent browser users (n = 606)*

Strategy	Precision (all windows 1–60 s)	Coverage
`HASH_AND_TIME_WINDOW`	100%	100%
`TIME_WINDOW`	89.93%	100%

Both strategies also coincide when broken down by local login density, including in the otherwise critical bucket below 100 ms, where both achieve 138 of 138 matches. In this dataset *usernameHash* never once excludes a candidate that *ipHash* had not already ruled out on its own, because every browser container had a stable address of its own and each *ipHash* bucket was therefore already unambiguous.

This shows that *ipHash* separates distinct sources, whether different devices, networks or, as here, containers, just as effectively as the full combination of both identifiers. It does not show what happens when several users share one address, as is the case behind corporate NAT or on a public wireless network. That scenario could not be produced with the existing test setup, which assigns one container per user, and is examined in the following section.

6.2.3.5 Precision among Multiple Users Sharing One Address

The preceding section established the contribution of *ipHash* only for separate sources. The opposite case, several users behind a shared address, is at least as relevant in practice and could not be reproduced with the container based setup. A dedicated measurement tool was therefore developed that runs multiple users as parallel threads within a single process rather than in separate containers. Since all threads send from the same machine, they necessarily share a source address, which follows from the architecture rather than from any network configuration.

Before the evaluation, a cardinality aggregation on *ip_hash* over the period of the run confirmed exactly one distinct value across all 600 documents. The shared address is therefore measured rather than assumed. The data source is three test users with 200 logins each, started from the same process. The results are summarised in Table 6.14 and Table 6.15.

`HASH_AND_TIME_WINDOW` remains at 100% despite the shared address, since *usernameHash* carries the entire separation on its own, irrespective of whether *ipHash* contributes anything.

Table 6.10: *Precision by local login density under three concurrent browser users, window = 10 s*

Density Bucket	n	HASH_AND_TIME_WINDOW	TIME_WINDOW
< 100 ms	138	100%	55.80%
100–300 ms	132	100%	100%
300 ms–1 s	258	100%	100%
> 1 s	78	100%	100%

Table 6.11: *Comparison with the synthetic multi-user dataset*

	Synthetic (n = 600)	Real browser (n = 606)
Precision in the < 100 ms bucket	59.69%	55.80%
Share of logins below 100 ms	86.8% (521/600)	22.8% (138/606)
Overall precision of TIME_WINDOW	65.00%	89.93%

The diagnostic strategy of the preceding section serves as a cross-check and yields exactly the same figures here as TIME_WINDOW, identical for every window size and every density bucket at 73.67% precision with 442 matches and 158 incorrect assignments. This is not a coincidence but the necessary consequence of a single shared address. A filter for the only *ipHash* value occurring in the entire period excludes no candidate that would not already fall within the time window. In this dataset *ipHash* therefore demonstrably contributes nothing to the separation, which is the exact counterpart to the preceding section, where IP_ONLY coincided with HASH_AND_TIME_WINDOW because the address alone already sufficed.

The precision of TIME_WINDOW at 73.67% falls between the 65.00% measured in Section 6.2.3.1 and the 89.93% measured in Section 6.2.3.3, which is consistent with the observed density. 504 of 600 logins, or 84%, lie less than 100 ms apart, close to the 86.8% of the synthetic harness and well above the 22.8% of the browser based measurement. Several threads running without waiting times from a single process therefore generate a density comparable to the serial HTTP harness rather than to real browser interaction. The effect stems primarily from login density and not from the shared address itself, which is consistent with the conclusion already drawn in Section 6.2.3.3. The offset between report reception and event time ranges from −50 to +110 ms with a median of −26 ms (n = 600), which is of the same order of magnitude as in the other measurements.

6.2.4 Coverage and Operating Limits of the Correlation

Before turning to precision, the coverage of the correlation is considered, that is the proportion of post authentication reports for which a counterpart was found at all. The value is only determinable because the sweep described in Section 5.3.2 marks expired reports as permanently uncorrelated. Without that distinction, a report still awaiting its audit entry would be indistinguishable from one that will never receive it, and the denominator of the ratio would change with every query rather than remaining fixed.

Table 6.12: *Precision of `HASH_AND_TIME_WINDOW` with and without the usernameHash (n = 606)*

Strategy	Precision (all windows)	Candidates (±1 s)
`HASH_AND_TIME_WINDOW`	100%	1.00
Without *usernameHash*	100%	1.00

Table 6.13: *Mean candidate count per window (n = 606)*

Window	`HASH_AND_TIME_WINDOW`	Without *usernameHash*
±1 s	1.00	1.00
±2 s	2.19	2.19
±5 s	3.86	3.86
±10 s	7.49	7.49
±30 s	21.67	21.67
±60 s	41.58	41.58

Coverage is also independent of the strategy that produced a match. In every measurement reported above it remains at 100% from a window of ±5 s onwards, including for `TIME_WINDOW`, which loses precision but not coverage under high login density. The two properties therefore fail independently of one another. A weaker strategy does not leave reports unassigned, it assigns them to the wrong counterpart.

The coverage is determined on the performance series, the largest dataset and the one generated with the final configuration of the system. Over a period of about five hours, it generated 7,236 post authentication reports with both interceptors active, in bursts of 200 logins at a peak rate of roughly three logins per second, in the asynchronous as well as in the synchronous transmission mode. For every report, the corresponding outbound report existed as well, and the baseline runs produced neither of the two, which confirms that the configurations differed as intended. Every report was correlated with its audit entry through the inbound message identifier, and none expired without a counterpart. The coverage was therefore 100 % in this period, with a lower bound of 99.95 % for the 95 percent Wilson interval [60]. The load is synthetic, since it stems from a single user and a single SP, and it does not reach the density at which the heuristic strategies lose precision. The exact strategy, however, remained unaffected by it, as expected from its construction.

During the same period the detection layer raised 21,712 alerts for 10,854 audit entries. For every entry it raised one `UNENCRYPTED_ASSERTION` alert, since the benchmark addresses the deliberately weakly configured SP, and one `SUSPICIOUS_USER_AGENT` alert, since the benchmark client is not a browser, in addition to four configuration alerts. Every audit entry was therefore evaluated by the rule engine. The 7,236 post authentication reports correspond to the audit entries of the asynchronous and synchronous runs, whereas the remaining 3,618 entries stem from the baseline runs, in which the interceptors are not active. None of the alerts was a `FIELD_MISMATCH` or `CORRELATION_DISCREPANCY` alert. The discrepancy rule raises an alert exactly for reports that expire without a counterpart, and the consistency check raises one for every pair that disagrees, so that neither rule

Table 6.14: *Precision of both strategies with three users sharing one source address (n = 600)*

Strategy	Precision (all windows 1–60 s)	Coverage
HASH_AND_TIME_WINDOW	100%	100%
TIME_WINDOW	73.67%	100%

Table 6.15: *Precision by local login density with a shared source address, window = 10 s*

Density Bucket	n	HASH_AND_TIME_WINDOW	TIME_WINDOW
< 100 ms	504	100%	68.65%
100–300 ms	96	100%	100%
300 ms–1 s	0	—	—
> 1 s	0	—	—

produced a false alarm under this load, while the alerting itself remained operational throughout.

Window size is not the limiting factor. Precision remains at 100% even though the mean number of candidates rises from 6.4 at ±1 s to 197.6 at ±60 s. What matters is solely that the temporal jitter is smaller than the interval between two consecutive logins of the same combination of subject, address and SP.

This yields a quantifiable operational limit. With a jitter span of 173 ms and an observed login interval of approximately 330 ms, the limit lies at about six logins per second per tuple of subject, address and SP. Beyond this point the heuristic can no longer resolve the order reliably, whereas the exact strategy is unaffected by it. The incorrect assignments measured in Table 6.6 set in below an interval of 100 ms, which corresponds to about ten logins per second. The figure of six logins per second is the more conservative bound, since it follows from the full span of the observed offset rather than from the density at which errors actually occurred.

The measurements also support a concrete configuration recommendation, namely *lg.correlation.window-seconds=1*. Coverage and precision remain at 100%, but 6.4 candidates are examined per report instead of 58.1, which reduces the effort by a factor of nine on the basis of evidence rather than a fixed assumption.

The fallback chain has a quantifiable value. Where the unique identifier is absent from the audit log, for instance because it is not recorded for data protection reasons, the heuristic delivers equivalent results under the measured conditions. The loss of information is therefore not qualitative but quantitative, and it only takes effect above the login density stated above.

The cost of omitting the hashes is thus quantified rather than merely postulated. This applies, however, only to the comparison between the exact strategy and HASH_AND_TIME_WINDOW. If both *ipHash* and *usernameHash* are missing, so that TIME_WINDOW has to step in, the picture changes. Section 6.2.3.1 shows a drop in precision from 100% to 65% with three simultaneously active users, concentrated on logins less than 100 ms apart. For an SP with several concurrent users, TIME_WINDOW is therefore to be classified as a stopgap for the case of missing hashes rather than as an equivalent alternative, unlike what the

single user measurement in Section 6.2.3 alone would suggest, where both strategies are indistinguishable.

There is a first indication that the operational limit is not reached in practice. Section 6.2.3.2 measures the login density under real browser traffic instead of the synthetic HTTP client harness, and all 202 logins lie more than one second apart, well below the limit of roughly six logins per second calculated above. For a single automated browser user the synthetic worst case of Section 6.2.3 is therefore considerably denser than real timing, although this has not yet been shown for several concurrent real users.

What matters is the distribution of density, not merely the possibility of a collision. Section 6.2.3.3 shows that the weakness of `TIME_WINDOW` in the sub-100 ms zone measured in Section 6.2.3.1 is just as pronounced under real traffic, at approximately 56% precision, and is therefore not a measurement artifact. What differs is how often that zone is reached. Real concurrent users produce it noticeably less frequently, in 23% of logins, than the artificially dense stress case, which reaches it in 87%. For operation this means that the risk is real and reproducibly demonstrated, but occurs less often under the observed user load than the stress case figure alone would suggest.

The contribution of *ipHash* depends entirely on whether the source actually separates users. Section 6.2.3.4 shows that with separate sources *ipHash* alone separates just as well as *ipHash* and *usernameHash* combined, yielding identical candidate sets in every measured window. Section 6.2.3.5 shows the counterpart, namely that where several users share one address *ipHash* contributes nothing at all, since `HASH_AND_TIME_WINDOW` without *usernameHash* and `TIME_WINDOW` produce identical figures there. For scenarios in which *usernameHash* is unavailable, this is the differentiated answer. The heuristic is only as good as the least discriminating identifier available to it. Where a shared address and a missing user identifier coincide, only `TIME_WINDOW` remains, with the density dependence observed consistently throughout.

The productive strategy withstands the shared address case as long as *usernameHash* remains available. This is the practically most relevant conclusion of Section 6.2.3.5, since `HASH_AND_TIME_WINDOW` retains 100% precision although all three test users share one address. Only when the user identifier is absent in addition to the shared address does the correlation become a question of the time window alone, which reduces to the density dependence already established rather than constituting a new risk specific to network address translation.

6.3 Detection

This section evaluates the fifteen detection rules described in Section 5.3.3. It first establishes what the reported figures can and cannot show, then presents the test coverage per rule, and finally reports three defects that the evaluation itself uncovered.

6.3.1 What the Figures Represent

An important qualification precedes the results. The figures reported here are not the outcome of a sampling study over productive traffic. No labelled dataset of real logins

exists against which precision and recall could be computed. What is evaluated instead are test cases, that is scenarios constructed by the developer in which the expected outcome is fixed in advance.

Three terms are used throughout this section. A true positive proof means that a test constructs a scenario in which a rule is supposed to fire and the assertion confirms that it does. A true negative proof means that a test constructs a scenario in which no alert is supposed to be raised and the assertion confirms that none is. A false negative class is not a measured rate but a case derivable from the code in which an actual attack would structurally produce no alert.

The strength of a true negative proof is not uniform, and this distinction turns out to matter. A negative case constructed directly in Java verifies only that the rule remains silent when given a correctly formed input object. It does not verify whether the pipeline preceding the rule, comprising the Logstash parsing and the field mapping, produces such an object correctly in practice. A negative case established through a real login verifies both at once. Section 6.3.4 presents a case in which precisely this difference made a defect visible that had remained invisible otherwise.

This evaluation therefore answers whether the implemented code behaves exactly as intended for the constructed cases. It does not answer how often the system errs in operation, since that would require a comparison against traffic whose correct classification is known independently of the rules themselves. Section 6.5 returns to this distinction.

6.3.2 Test Layers

The rules are verified on three layers that differ in how much of the system they exercise. Unit tests examine a rule in isolation with its repository replaced by a mock. They verify the detection logic itself and are the fastest to write and to execute.

Layer one tests run the rule against a real Elasticsearch instance provided through Testcontainers. Compared with a unit test, this additionally covers the query as it is actually formulated in the Elasticsearch query language, the index mapping and the serialisation of the stored documents. The rule under test is invoked directly rather than through the scheduler, which decouples the correctness of the detection from the timing of the scheduler.

Layer two tests exercise the complete path. A real browser performs a login against the real IdP, the resulting audit entry passes through Filebeat and Logstash into Elasticsearch, and the test then verifies the alert produced by the backend. This is the only layer on which a defect in the log shipping pipeline can become visible.

Two of the layer two tests are false positive runs. Each performs 100 consecutive logins of three rotating users through the browser and examines the alerts that arose in the observed period. Against the clean SP the assertion is that no alert of any type was raised, and against the weakly configured provider the assertion is that no `FIELD_MISMATCH` alert was raised, since the alerts of the other rules are expected there. Both runs wait about 100 seconds after the last login, so that alerts that arise only after the retry window of the correlation, such as the discrepancy alert, are not covered by them.

6.3.3 Coverage per Rule

Table 6.16 shows which layers each rule is covered on.

Table 6.16: *Test coverage per rule across the three layers*

Alert Type	Unit	Layer 1	Layer 2
SUSPICIOUS_USER_AGENT	yes	yes	TP, TN
FAILED_LOGIN	yes	yes	TP, TN
BRUTE_FORCE	yes	yes	TP, TN
UNENCRYPTED_ASSERTION	yes	yes	TP, TN
WEAK_ENCRYPTION	yes	yes	TN only
FIELD_MISMATCH	yes	yes	TN, TP (manual)
CORRELATION_DISCREPANCY	yes	yes	—
BEHAVIOUR_ANOMALY	yes	yes	—
MFA_BYPASS	yes	yes	—
ASSERTION_REPLAY	yes	yes	—
WEAK_CONFIGURATION	yes	yes	—
MISSING_MFA	yes	yes	—
CERTIFICATE_EXPIRED	yes	yes	—
SIGNATURE_WRAPPING_SUSPECTED	yes	yes	—
REQUEST_REPLAY	yes	yes	—

All fifteen rules are covered by an explicit true positive proof against a real Elasticsearch instance. The counts comprise 118 unit tests for the fifteen rules, 21 layer one tests and seven layer two tests, counted as annotated test methods. The unit tests of the four rule engines, 19 further tests, are not included. Of the layer two tests, four are true positive tests for individual rules, one is a baseline with a single clean login for five rule types, and two are the false positive runs.

Six rules are additionally covered on layer two. Four of them, namely SUSPICIOUS_USER_AGENT, FAILED_LOGIN, BRUTE_FORCE and UNENCRYPTED_ASSERTION, have both a true positive and a true negative proof. WEAK_ENCRYPTION has only a true negative proof at present, and FIELD_MISMATCH has an automated true negative proof in the 200 logins of the false positive runs, whereas its true positive proof is the manual manipulation described in Section 6.3.7. Because the clean run asserts that no alert of any kind occurred, it also serves as a true negative proof for the remaining rules, insofar as they can raise an alert within the observation period.

The remaining nine rules are not covered on layer two, for three different reasons. Five of them, namely BEHAVIOUR_ANOMALY, MFA_BYPASS, ASSERTION_REPLAY, SIGNATURE_WRAPPING_SUSPECTED and REQUEST_REPLAY, cannot sensibly be triggered through a normal login, since they require a manipulated message or an accumulated history rather than a single successful authentication. The other three, namely WEAK_CONFIGURATION, MISSING_MFA and CERTIFICATE_EXPIRED, do not depend on a login at all. They evaluate configuration and metadata, which the layer one tests already exercise in full, so that layer two would add nothing for them. The ninth, CORRELATION_DISCREPANCY, raises its alert only after the retry window of fifteen minutes, which the end to end tests do not wait for.

6.3.4 A Defect Visible Only through the Full Pipeline

The true negative proof for `UNENCRYPTED_ASSERTION` on layer two required a service provider whose assertions are actually encrypted, since the existing one transmitted them in plain text. A second provider was therefore registered with `encryptAssertions=true` and a real login performed against it. The expectation was that no alert would be raised.

The first run contradicted this. For the same login, Elasticsearch contained both *encryption_algo* with the value `AES128-GCM` and *encrypted* with the value `false`, which is an internal contradiction since both fields originate from the same audit line.

The cause lay in the log shipping pipeline rather than in the rule. The audit format logs `%XA`, the algorithm, which Logstash maps correctly. It does not log the separate standard token for the encryption state, which Logstash therefore derives from a backward compatibility alias that the audit format never populated. The subsequent processing logic recognises only the literal strings `true` and `false`, so that an empty value falls into the same branch as `false`. The field *encrypted* was consequently set to `false` for every login, entirely independently of the actual encryption state.

The consequence is not a missed detection but the opposite. For genuinely unencrypted logins the result remained accidentally correct. For encrypted logins the rule would have reported every single one as `UNENCRYPTED_ASSERTION`. This is not a rare edge case but a structurally guaranteed false positive rate of one hundred percent across all encrypted traffic, concealed solely by the fact that the test environment had until then contained no SP with encryption enabled.

The defect was invisible on layer one for a structural reason. The three existing negative scenarios for this rule construct their audit object directly in Java with the encryption state set explicitly and therefore never pass through Logstash at all. A test that bypasses the pipeline cannot, in principle, find a defect located within it. The fix derives the encryption state from the algorithm field, treating a non-empty value as encrypted.

After restarting Logstash, a login against the same SP produced documents carrying *encryption_algo* with the value `AES128-GCM` and *encrypted* correctly set to `true`.

6.3.5 A Defect Visible Only against Real Persistence

A second defect illustrates why layer one is not redundant with respect to unit testing. During development, the unit test for `BEHAVIOUR_ANOMALY` passed consistently while the rule did not work in operation. The behaviour profile of a user was written once and never updated thereafter. The cause was a missing version field on the stored document, which caused Elasticsearch to discard every update silently rather than to reject it with an error.

A mock-based test cannot detect this class of defect. The mocked repository returns success for every save operation, irrespective of whether a real Elasticsearch instance would have accepted the document. The test therefore verified that the rule calls the repository, not that the repository call has any effect. This finding is the reason why layer one was introduced for all rules rather than only for those where it seemed most obviously necessary.

6.3.6 A Systemic Class of False Negatives

Seven of the fifteen rules begin by checking whether the audit field they depend on is present in the field registry, and return without a result if it is not, and the consistency check skips each field that the registry does not contain. As a design decision this is sound, since a rule should not guess when the data it requires is absent, as argued in Section 4.5.2.

The consequence, however, is that the detection coverage of the system is bounded from above by the completeness of the audit configuration rather than by the detection logic itself. An operator who shortens the audit format for reasons of data protection or storage reduces the detection surface accordingly, and no alert or warning indicates that this has happened. The rules do not fail in that case, they simply remain silent.

A related case occurred in the development environment of this work. Two rules depend on the SP configuration, which reaches the backend exclusively through a dedicated ingest path. That path was inactive for an extended period, first because a security configuration no longer permitted the corresponding request and later because the pipeline configuration referenced an outdated address. Both rules therefore evaluated against an index that was no longer being updated, without any indication that their input had become stale.

6.3.7 Consistency between Outbound Report and Audit Entry

The consistency check described in Section 4.5.2 compares the outbound report of a login with the audit entry that the post authentication report of the same login was correlated with. It runs after the correlation and compares the fields that both records contain and that the audit format of the deployment records. For every login these are the status, the identifier of the request answered by the response, the identifier and the issue time of the outbound message and the encryption state. Where the assertion is not encrypted, the identifier of the assertion, the authentication instant, the authentication context, the name identifier format and the pseudonymised name identifier are compared as well. Values that the audit log writes in an abbreviated form are mapped to the notation of the outbound report before the comparison.

The evaluation addresses two questions. The first is whether the check remains silent for legitimate logins. To examine this, 100 consecutive logins of three rotating test users were performed through a real browser against each of two SPs, namely the deliberately clean provider with encrypted assertions and the weakly configured provider without encryption described in Section 6.1. The IdP transmitted its records in the asynchronous mode. Every login ran through the complete pipeline, from the browser to the comparison in the backend. Table 6.17 summarises the result.

Table 6.17: *Result of the consistency check for legitimate logins (upper bound: 95 percent Wilson interval)*

SP	Logins	Fields compared	Checks passed	Mismatches	Upper bound
Clean SP	100	5	100	0	3.7 %
Weak SP	100	10	100	0	3.7 %

In all 200 logins the post authentication report was correlated with its audit entry through the inbound message identifier, no report expired without a counterpart, and the check passed. No `FIELD_MISMATCH` alert was raised. The number of compared fields is taken from the log of the check, which lists the fields of every comparison. It is five for the clean provider and ten for the weak one in each of the 100 logins. With no deviation in 100 comparisons per provider, the upper bound of the 95 percent Wilson interval for the false positive rate of the check is 3.7 percent for each provider and 1.9 percent for both together. The two runs differ in the fields that can be compared. For encrypted assertions the assertion internal fields are not observable at the point where the outbound interceptor sees the response, so that only the response level fields and the encryption state take part. The run against the weak provider additionally covers the assertion internal fields, including the pseudonymised name identifier, and with them the mapping of the values that the audit format abbreviates. Earlier runs in the synchronous transmission mode produced the same result. The performance series described in Section 6.4 adds 7,236 further pairs of records, for which no `FIELD_MISMATCH` alert was raised either (Section 6.2.4).

The second question is whether the check reports a discrepancy that actually exists. For this purpose a login against the clean provider was performed while the backend was stopped. After the audit entry had reached Elasticsearch, one of its fields, the identifier of the request answered by the response, was replaced by a different value, and the backend was started again. At its next correlation run the backend matched the post authentication report to the audit entry through the unchanged inbound message identifier, within two seconds of the start. The check then raised a `FIELD_MISMATCH` alert of high severity that names exactly this field, and no other field of the pair was reported.

This demonstrates that the mechanism detects a contradiction between two independently produced records of the same event. It does not provide a detection rate, since it rests on a single manipulation of a single field. The modification was applied to the stored audit entry and not to the log file on the IdP. Because the outbound report is produced independently of that file, the check would react in the same way to a modification of the file before it is shipped, but this case was not demonstrated separately. Finally, the check establishes that the records disagree and not which of them was altered. An attacker who controls the application process can manipulate the report as well, as discussed in Section **??**.

The check is triggered by the correlation of the post authentication report. Events that only the outbound interceptor observes, such as an error response that the IdP issues before authentication, are therefore not compared with their audit entries, although the outbound report and the audit entry would suffice for such a comparison.

6.3.8 Coverage of the Documented Attack Space

The figures presented so far describe the relationship between the rules and the tests written for them. They say nothing about the relationship between the rules and the attacks that actually occur, since both the rules and the scenarios originate from the same source.

A partial answer is available through the assignment established in Section 4.5.2, where the rules were related to the vulnerabilities and attack classes documented by Simpson and Groß [52]. That assignment provides a denominator that was not defined by this work. Of the six cross-protocol issues concerning SAML, the implemented rules address two directly, while a third, namely `SIGNATURE_WRAPPING_SUSPECTED`, addresses the same attack class through a vulnerability that the survey does not list for SAML.

The remaining issues are not addressed, and the reasons are structural rather than a matter of implementation effort. They are discussed in Chapter **??**.

6.3.9 Thresholds

Several rules depend on numerical thresholds, among them the number of failed attempts and the length of the window within which they are counted, the number of logins before a behaviour profile is considered established, and the number of occurrences after which a replay is reported.

None of these values has been subjected to a sensitivity analysis, that is an examination of the threshold at which a true positive turns into a false negative. They were chosen as plausible point values rather than derived from measurement. Since the thresholds were moved into the configuration during this work, such an analysis has become technically possible, but it has not been carried out.

6.3.10 Fidelity of the Rules and Defects Uncovered by Testing

All fifteen rules behave as intended for the scenarios constructed for them, and all are verified against real persistence rather than against mocks alone. This is a statement about the fidelity of the implementation, and within that scope it is a complete one.

It is not a statement about detection quality in operation. Determining how reliably a rule identifies genuine attacks would require productive traffic in which each login is independently labelled as malicious or benign. Such a dataset does not exist for this work and cannot be produced within a test environment. The circularity of rules and scenarios originating from the same source is therefore not resolved by the figures reported here, although the derivation of the rules themselves, as described in Section 4.5.2, no longer rests on the author's own judgement.

What the evaluation of the rules did establish is of a different kind. Three defects became visible that operation itself did not reveal. A missing version field caused every profile update to be discarded silently, a pipeline defect would have reported every encrypted login as unencrypted, and the dependence of two rules on a stale index went unnoticed for months. In all three cases the system continued to run, alerts continued to be produced and the dashboard continued to be populated. The defects became apparent only when the behaviour was measured systematically against an expectation defined in advance.

6.4 Performance Overhead

This section presents the results of the overhead evaluation of the two interceptors. Both run inside the IdP and therefore lie on the critical path of each authentication process, the first as a *postAuthenticationFlow* and the second as an *outboundInterceptorFlow*. The evaluation addresses two questions:

- How much overhead do the interceptors introduce to the end to end authentication latency and to the processor load of the IdP?
- Does the difference between asynchronous and synchronous transmission justify the use of fire and forget transmission?

Three configurations are compared, namely *baseline*, *async* and *sync*. In the *baseline* configuration, neither of the two interceptor flows is configured for the SP, so that no interceptor is executed during the authentication process. In the *async* configuration, both flows are enabled, and each interceptor transmits its data to the monitoring tool without waiting for a response, in the fire and forget manner. In the *sync* configuration, both flows are enabled as well, but each interceptor waits for the response of the monitoring tool before it completes, so that a login contains two blocking transmissions.

Comparing *baseline* against *async* quantifies the cost of the interceptors in their productive configuration. Comparing *async* against *sync* quantifies the cost of the design decision described in Section 5.2, namely what waiting for a confirmation would cost if the records were transmitted synchronously.

6.4.1 Measurement Sources

The overhead is evaluated using three measurements that differ in what they include. The *internal* measurement is obtained through the *executionTimeMs* instrumentation within each of the two interceptors. It measures the time required for data collection, hashing and the extraction of fields performed by the action. This represents a lower bound, because payload construction and data transmission are excluded. The measured value is part of the payload and is therefore available for every login without additional instrumentation.

The *external* measurement records the end to end duration of a login as observed by the client. It includes everything the user experiences, from the initial request to the point at which the SP grants access, and therefore covers the interceptors together with all other processing steps of the IdP. This is the measurement that answers whether the overhead is perceptible in practice.

The *CPU* measurement records the processor utilisation of the IdP container during a run via `docker stats`, sampled once per second. It serves as an independent confirmation, since the two latency measurements could in principle fail to detect an effect that manifests itself as additional computation rather than as additional waiting time.

6.4.2 Experimental Design

The experiment comprises 18 runs, each executing all three configurations in sequence, which results in 54 measurements in total. Each measurement consists of one hundred

logins whose latency is recorded, preceded by one hundred warm-up logins whose values are discarded. Every login is performed through the HTTP login harness described in Section 6.1 with a fresh client and an empty cookie store, so that each login passes through the full authentication path and no existing session is reused.

The order of the configurations is rotated between runs. Six different orders are used, and the series repeats them three times, so that each configuration occupies each of the three positions six times. Table 6.18 shows the six orders, which apply to runs 1 to 6, 7 to 12 and 13 to 18.

Table 6.18: *Order of configurations in runs 1 to 6, repeated in runs 7 to 12 and 13 to 18*

Run	Position 1	Position 2	Position 3
1	baseline	async	sync
2	sync	async	baseline
3	async	sync	baseline
4	sync	baseline	async
5	baseline	sync	async
6	async	baseline	sync

This balancing was necessary because a position drift occurs within a session, which is examined in Section 6.4.6. Without rotation, a configuration measured consistently in the last position would appear slower than one measured first, regardless of its actual cost.

For each measurement the IdP container is recreated with the settings of the configuration that is measured, baseline, async or sync. The monitoring backend runs during all measurements, including the baseline, so that the environment is identical, and a pause of four minutes follows each measurement so that the backend can process the records that were generated. The series took about five hours.

The experimental unit is the run, not the individual login. The one hundred logins within a run share the same JVM state, the same container and the same system load, and are therefore pseudo-replicates rather than independent observations. Treating them as independent would suggest a precision that the experiment does not provide, since the dominant source of variation lies between runs rather than within them. Each run consequently contributes a single value per configuration, namely the median of its one hundred logins, which yields 54 observations for the analysis, 18 per configuration.

Before each measurement, a single test login verifies that the intended configuration is in effect. The verification queries the monitoring backend and checks whether the login produced a post authentication report and an outbound report, expecting none of them for *baseline* and one of each for the other two configurations. This gate was introduced after an earlier series had to be discarded, in which the backend had not been reachable and all three configurations had therefore been measured under effectively identical conditions.

6.4.3 Descriptive Results

Table 6.19 shows how long a login takes in each configuration. The p50 is the median login, that is the duration that half of the logins stay below, and the p75 and p90 describe

the slower logins, since 25 and 10 percent of the logins take longer than these values. Each run contributes the percentiles of its 100 logins, and the table shows their average over the 18 runs.

Compared with the baseline, the median login takes 6.2 ms or 1.7 percent longer in the asynchronous configuration and 59.6 ms or 16.7 percent longer in the synchronous configuration. For the synchronous configuration the difference grows towards the slower logins, from 59.6 ms at the p50 to 92.0 ms or 21.6 percent at the p90. The asynchronous configuration, in contrast, stays within 6 to 10 ms of the baseline over the whole distribution. Blocking transmission therefore adds a clearly visible amount to every login, whereas the asynchronous mode stays close to the baseline. Percentiles are reported rather than means, because a few slow logins pull the mean above the median.

These raw differences do not yet show whether they exceed the variation between runs. This variation is described by the standard deviation of the 18 run medians of a configuration, which lies between 14.9 and 19.2 ms and is therefore larger than the difference between the baseline and the asynchronous configuration. Section 6.4.4 tests the differences against this variation.

The interceptors report how long their own work takes. The median is 0.415 ms for the post authentication interceptor (95th percentile 1.16 ms, 99th percentile 1.83 ms) and 0.039 ms for the outbound interceptor (0.14 ms and 0.30 ms), determined from all reports stored by the backend. Together this is about 0.45 ms, or 0.13 percent of the median login duration of the baseline, and a lower bound, since payload construction and transmission are not included. It is far smaller than the difference of the synchronous configuration, which therefore stems from waiting for the backend and not from the work of the interceptors.

Table 6.19: *End to end login latency per configuration, mean of the run percentiles over 18 runs*

Configuration	p50	p75	p90
baseline	356.5 ms	390.5 ms	425.1 ms
async	362.7 ms	400.5 ms	433.2 ms
sync	416.1 ms	465.3 ms	517.1 ms

6.4.4 Paired Analysis of the Latency

The raw configuration means still contain two known sources of variation, namely the difference between runs and the position drift within a run. Although the design balances both, the values themselves remain affected by them. A linear model with configuration as a factor and run and measurement position as block factors accounts for these effects explicitly before the configuration difference is estimated, which narrows the confidence intervals and permits a fair comparison even where individual runs happened to be slower or faster. The model has 22 parameters and is fitted to 54 observations, which leaves 32 degrees of freedom.

The two comparisons that involve the synchronous configuration are significant at the five percent level, whereas the comparison between the asynchronous configuration and

Table 6.20: *Paired comparison of the login latency, residual SD 16.0 ms, df = 32*

Comparison	Difference	95% CI	t	Direction
async – baseline	+6.2 ms	[−4.7; +17.1]	+1.16	12/18 positive
sync – baseline	+59.6 ms	[+48.7; +70.4]	+11.17	18/18 positive
sync – async	+53.4 ms	[+42.5; +64.2]	+10.01	18/18 positive

the baseline is not, since its confidence interval includes zero. The direction is consistent for the synchronous configuration against both other configurations, which is positive in all 18 runs.

The direction column is not part of the model but a simple check. For each run the raw paired difference between two configurations measured within that same run is computed, and the number of positive differences is counted. A consistent direction across runs is an indication that the effect is real, whereas an effect that changes sign between runs is more likely to be noise.

6.4.5 CPU Utilisation

The CPU measurement records the utilisation of the IdP container, where 100 % corresponds to one fully loaded core. Sixteen cores were available and no resource limits were configured. The startup phase preceding each measurement, during which class loading and just in time compilation produce peaks of up to 400 %, was excluded on the basis of the start time recorded in the corresponding benchmark file. Table 6.21 shows the aggregated utilisation per configuration.

Table 6.21: *CPU utilisation of the IdP container per configuration in percent of one core*

Configuration	Mean (SD)	Median	Min	Max
baseline	34.9 (2.7)	32.7	16.3	95.7
async	41.9 (3.3)	39.6	15.2	123.4
sync	39.2 (3.5)	35.3	14.6	126.7

Mean and standard deviation refer to the 18 run means per configuration, the median to the 18 run medians, whereas minimum and maximum refer to all samples of that configuration. The comparison between configurations uses the median of each run, since the samples contain peaks of up to 127 % that distort a mean. The same block model as for the latency was applied.

Table 6.22 shows the result. The asynchronous configuration uses significantly more processor time than the baseline, and the synchronous configuration uses significantly less than the asynchronous one. The difference between the synchronous configuration and the baseline is not significant.

Table 6.22: *Paired comparison of the CPU utilisation, median based, residual SD 4.4 percentage points, df = 32*

Comparison	Difference	95% CI	t	Direction
async – baseline	+6.9 pp	[+4.0; +9.9]	+4.72	16/18 positive
sync – baseline	+2.4 pp	[−0.6; +5.4]	+1.64	13/18 positive
sync – async	−4.5 pp	[−7.5; −1.5]	−3.08	5/18 positive

6.4.6 Position Drift as a Methodological Finding

The measurements also permit an observation about the measurement environment itself as shown in Table 6.23. The second position lies 8.5 ms and the third 10.3 ms above the first. Each configuration occupies each position six times, so that the means are directly comparable. The drift exceeds the point estimate of the asynchronous overhead of 6.2 ms. Had the configurations been measured in a fixed order, the position effect would therefore have been indistinguishable from that overhead, and whichever configuration occupied the last position would have appeared slower. Against the effect of the synchronous mode of 59.6 ms, the drift is small.

The balanced design prevents this confounding, although the drift still increases the residual variance. The rotation of the measurement order was therefore necessary rather than cosmetic, and this is arguably the most transferable finding of the performance evaluation, since it applies to any comparison of configurations measured within a shared session.

Table 6.23: *Mean latency by measurement position within a run*

Position within the run	Mean p50
1	372.2 ms
2	380.7 ms
3	382.5 ms

6.4.7 Resolution of the Measurement Environment

The valid series itself shows which effect sizes the environment can resolve. The standard deviation of the 18 run medians of a configuration lies between 14.9 and 19.2 ms, and the residual standard deviation of the block model is 16.0 ms. The standard error of a contrast between two configurations is therefore about 5.3 ms, and with 18 runs per configuration the confidence intervals of the latency contrasts have a half width of about 11 ms. Differences of this size or larger become significant, whereas smaller ones cannot be distinguished from noise.

6.4.8 Correction of the Analysis Script

The analysis script of an earlier series of this experiment contained two indexing errors, among them a missing final measurement row per run during the computation of the means. Both were corrected before the present series was evaluated.

The errors are worth reporting because they were not visible in the output. The script produced plausible figures in both cases, and the discrepancy only became apparent when individual values were traced back to the raw measurement files. This mirrors the pattern observed in the correlation evaluation, where a defect likewise manifested itself not as an error but as a result that appeared correct. A further off by one error, which would have omitted the last run in four functions of the script, was found by inspection before the present series was analysed. The analysis scripts now check that the number of values matches the number of runs and stop otherwise.

6.4.9 Latency and Processor Cost of the Interceptors

The asynchronous interceptors introduce no latency overhead that the experiment can distinguish from zero. The point estimate is +6.2 ms, which is 1.7 percent of the median login duration of the baseline, and the upper bound of the 95 percent confidence interval is 17.1 ms, or 4.8 percent. In twelve of the 18 runs the asynchronous configuration was slower than the baseline, which is a weak tendency but not an established effect. The internal measurement attributes about 0.45 ms to the work performed within the two interceptors, so that a difference of several milliseconds would have to originate elsewhere, for instance in the threads that perform the transmission. The claim is therefore a bounded one. Differences of about 11 ms and more would have been detected, and the overhead of the asynchronous mode lies below roughly five percent of the login duration.

The asynchronous mode is not free of cost, however. It increases the processor utilisation of the IdP container by 6.9 percentage points of one core, which is about 21 percent relative to the baseline and 0.4 percent of the capacity of the 16 available cores. This is consistent with the nature of the two modes. Asynchronous transmission moves the work into background threads, where it appears in the processor utilisation rather than in the latency, whereas synchronous transmission spends the same time waiting for a response. The synchronous configuration consequently lies between the other two. Its difference of 2.4 percentage points to the baseline cannot be distinguished from noise, whereas its difference of 4.5 percentage points to the asynchronous configuration is significant.

The synchronous mode adds 59.6 ms to every login, which is 16.7 percent, and the difference is established in all 18 runs. Since the configuration contains two blocking transmissions, this corresponds to roughly 27 to 30 ms for each transmission, assuming that both cost the same.

For the design decision described in Section 5.2, the measurement therefore supports the asynchronous mode with a statistically established difference. Blocking transmission would add about 60 ms to every login, whereas the asynchronous mode stays below the resolution of the experiment in latency and pays with a moderate increase in processor

load. The results apply to the test environment and to the limits discussed in Section 6.5, namely a single host, an HTTP client instead of a browser and a single SP.

6.5 Threats to Validity

The preceding sections reported what the measurements show. This section states what they do not show, and why. The limitations are grouped by their origin, namely the measurement environment, the traffic used, the statistical basis, and the questions that the chosen setup cannot answer at all.

6.5.1 Measurement Environment

The design described in Chapter 4 places the monitoring backend outside the trust boundary of the IdP, which in a productive deployment would imply separate hosts under separate administrative control. The test environment does not realise this separation physically. All components run in containers on a single machine, which reproduces the logical separation of the components but not the security property derived from it. The measurements are therefore valid with respect to latency and correlation behaviour, but the environment does not demonstrate the trust boundary itself.

The absolute values are therefore not transferable to a productive deployment. A login duration of approximately 360 ms reflects this particular setup and would differ on separated hosts, under a different network topology or with a different user store. The differences between configurations are transferable to a greater degree, since they were measured in pairs within a single session and the shared conditions cancel out in the comparison.

The backend remained active during all runs, including the *baseline* configuration in which the interceptors is not executed. This was deliberate, so that the background load would be identical across configurations. It also means that the measurements do not reflect a deployment in which no monitoring infrastructure is present at all.

6.5.2 Traffic and Scope

Most measurements rest on a single test user, a single SP and a single authentication flow, namely password-based authentication. For the correlation this represents the hardest case with respect to candidate ambiguity, since all records share the same subject, the same address and the same SP, leaving time as the only distinguishing feature. With respect to temporal regularity it is the easier case, since a synthetic client produces more evenly spaced logins than real users. In a productive network the jitter would be larger and the operational limit derived in Section 6.2.4 correspondingly lower. The offset statistic reported there is the figure by which this can be measured again in a different environment.

Several measurements use an HTTP client harness rather than a browser. Sections 6.2.3.2 and 6.2.3.3 address this by repeating the measurement against real browser traffic, which

showed the density effect to be reproducible while occurring considerably less often under realistic timing.

Single sign-on follow-up requests, in which an existing session is reused without a full authentication, were not measured. The interceptor would carry relatively more weight there, since the total latency of such a request is lower.

6.5.3 Statistical Basis

The performance evaluation rests on 18 runs per configuration, which yield 54 observations. The 95 percent confidence intervals of the latency contrasts have a half width of about 11 ms, so that differences of this size or larger are established. The conclusion that the asynchronous mode introduces no measurable latency overhead is therefore a statement about an upper bound of about 17 ms, or five percent of the median login duration, and not about the absence of any effect.

Six significance tests were performed without correction for multiple testing, namely three comparisons for the latency and three for the processor utilisation. The four significant results remain significant under a Bonferroni correction for six tests, whose threshold is 0.0083, with the weakest being the processor comparison between the synchronous and the asynchronous configuration, for which $p \approx 0.004$ [2]. The two comparisons that are not significant are unaffected.

The correlation measurements each rest on a single run, with sample sizes between 202 and 606 reports. Precision is reported with Wilson score confidence intervals rather than the normal approximation, since the latter yields bounds outside the valid range or an interval of zero width where the observed precision reaches one hundred percent.

6.5.4 What the Detection Figures Establish

The detection evaluation is subject to a limitation of a different kind, which concerns not the precision of the figures but their meaning. Both the rules and the scenarios against which they are verified originate from the same source. A test that confirms a rule to behave as intended for a constructed case establishes the fidelity of the implementation. It does not establish that the rule detects genuine attacks, since the same understanding that produced the rule also produced the case.

This circularity is partially resolved and partially not. The derivation of the rules no longer rests on the author’s judgement, since the vulnerabilities and attack classes they address are taken from the survey by Simpson and Groß [52] and from the OASIS security considerations [21]. Which conditions are worth detecting is therefore determined externally. The verification of the rules, by contrast, remains internal.

Resolving it entirely would require productive traffic in which each login is independently labelled as malicious or benign. No such dataset exists for this work, and it cannot be produced within a test environment, since producing it would require the same judgement that the labelling is meant to verify.

A partial measurement is nevertheless available. In 100 consecutive logins of three users against the clean SP, no alert of any type was raised, which bounds the proportion

of legitimate logins that trigger an alert in this scenario to at most 3.7 percent (95 percent Wilson interval). The measurement covers only an SP whose configuration is free of the weaknesses the rules are designed to detect, it uses a single browser, and it does not reach alerts that arise only after the retry window of fifteen minutes. It says nothing about the behaviour against configurations that are legitimate but unusual, which is the harder case in a production deployment.

The false negative classes reported per rule are likewise derived rather than measured. Where a test deliberately carries a negative case, the statement is firm. Where a false negative class was inferred from the code path, it is plausible but not demonstrated by a failing test, since such a test would expose the same gap it describes.

The consistency check rests on a map that mirrors the abbreviations of the audit format of the IdP, for instance the notation of the status or of the authentication context. The backend holds a copy of the default map, which is not synchronised with the audit configuration of the IdP. Should an operator change the map there, the check would report deviations for values that are in fact consistent. The demonstration of the check rests on a single manipulation of a single field. It shows that the mechanism works but provides no detection rate.

A discrepancy alert cannot distinguish a counterpart that never arrives from one that arrives only after the retry window, for instance because of a delay in the log shipping chain. A report that has expired is not correlated again afterwards, so that such a late entry stays unmatched and the alert remains.

6.5.5 Derived but Not Validated

Two aspects of the system follow a documented derivation without having been validated empirically. The severity levels follow the risk classification of Graves as described in Section 4.5.3. That the rules distinguish correctly between the levels is verified in several tests. Whether the resulting classification prioritises appropriately in operation is not verified, since this would require feedback from operators on alerts actually raised.

The numerical thresholds of several rules are point values chosen as plausible rather than determined by measurement. No sensitivity analysis was performed, meaning that the threshold at which a true positive turns into a false negative is unknown. Since the values were moved into the configuration during this work, such an analysis has become technically possible but remains outstanding.

6.5.6 Known Gaps

Three specific gaps are worth stating explicitly rather than leaving to be inferred. `WEAK_ENCRYPTION` has a true negative proof on layer two but no true positive proof, since establishing one would require an SP configured to use a weak algorithm.

Nine of the fifteen rules are not covered by an explicit proof on layer two. As discussed in Section 6.3.3, five of them cannot sensibly be triggered through a normal login, three do not depend on a login at all, and the discrepancy rule raises its alert only after the retry window of fifteen minutes, which the end to end tests do not wait for.

For the correlation, the case in which a shared source address and a missing user identifier coincide was not measured. Section 6.2.3.5 covers the shared address with the user identifier present, where the latter carries the entire separation. The combined case reduces methodologically to the density dependence of `TIME_WINDOW` established throughout, rather than constituting an unknown of its own.